# 综合实战项目

# 报　告　书

**班级：2019 级数据智能分析方向**

**小组：第五组**

**组长：宋智琪**

**成员：冯姝月 李姝汶 苏路方 王泽涛 王志龙**

**日期：2022 年 4 月**

**1. 确定项目**.....3

1.1 项目名称.....3

1.2 项目概述.....3

1.3 项目说明.....3

1.4 运行环境.....4

**2. 项目实施**.....5

2.1 数据采集.....5

2.2 数据清洗.....12

2.3 数据分析.....16

2.4 数据导出迁徙.....35

2.5 数据可视化.....43

2.6 数据挖掘分析.....71

**3. 项目总结**.....82

**4. 源码附录**.....86

# 1.确定项目

## 1.1 项目名称

基于“58 同城”平台的上海二手房信息的决策分析报告

## 1.2 项目概述

2021 年是二手房市场迎来大变革的一年，二手房价格参考制度成为全新的调控工具，对市场交易和预期都产生巨大影响。作为国内领先的房产信息及交易服务平台，我们通过对 58 同城的二手房信息搜集整理，通过对数据进行数据清理、可视化以及通过数据挖掘对上海二手房进行预测,从住房者关注的多方面进行分析,为有意向的相关房客提供借鉴和参考。

该项目通过使用 Scrapy 爬虫框架、数据采集和数据清洗使用 Pandas 工具、MySQL 数据库、中文分词组件 Jieba、数据可视化框架以及一些代码格式化工具等，对“58 同城”的上海二手房的信息数据进行采集、清洗、分析最终生成 CSV 表的形式并导入到 MySQL 数据库中，同时，为了使最终得到的数据能够更加清晰直观，通过构建 Django 开发框架，使得导出到 MySQL 中的数据在页面上能够以图表的形式进行直观展示查看并进行分析。最后的数据挖掘部分通过构建出最佳的决策树回归分析模型以实现用户根据自身需求预测出大致房源总价，选择合适房源的目的。

## 1.3 项目说明

- 数据来源： 58 同城
- 通过爬取网站数据，对数据分析，来进行以下问题的探索：
  1. 上海二手房面积
  2. 上海二手房出售单价
  3. 上海二手房出售总价格
  4. 上海二手房所出售房子位置
  5. 上海二手房房子经纪人评分
  6. 上海二手房所出售房子的层数
  7. 上海二手房房屋建筑年份地区分布中排名前五城市的薪资等级分布
  8. 上海二手房总价预测分析
  9. 上海二手房推荐

- 具体步骤如下：
  1. 数据采集

首先要确定数据的来源,我们在这里选择的是对 58 同城进行爬取,主要通过 python 的 requests 模块进行爬取。

2. 数据清洗

数据清洗部分主要采用 Pandas 的 DataFrame，直接读取数据采集部分得到的 liangjia_shanghai.csv 文件,读取原始数据,删除掉几乎为空的以及后期不需要的字段列； 进一步清洗数据，将 houseLocation 的内容只取区名，舍弃具体地址；将 houseSinglePrice 以及 houseTotalMoney 的单位以便后续分析。数据分析

通过 pyspark 来分析后续可视化所需要的指标。以及通过 python 对数据进行词云统计分析。

3. 数据导出迁徙

通过 pyspark 来分析数据，接着将数据导入到 mysql 数据库中，要注意表名与数据库名的正确性。

4. 数据可视化

通过 Django MVT 模式与 echarts 来实现数据可视化，主要包括柱状图、地图以及词云等图的可视化。

5. 数据挖掘分析

通过 PySpark MLlib 决策树回归算法与朴素贝叶斯算法对 58 同城有关数据分析岗位的招聘信息探索，本项目首先通过朴素贝叶斯算法对招聘信息及其薪资进行分类，再通过决策树算法对不同情况下的薪资进行预测。

## 1.4 运行环境

Linux Ubuntu 16.04
PyCharm Community Edition 2019.3.2
Anaconda2-2.5.0-Linux-x86_64
hadoop-2.6.4
spark-2.0.0
IPython Notebook
mysql-5.7.24
jdk-7u75-linux-x64
scala-2.10.4

# 2.项目实施

## 2.1 数据采集

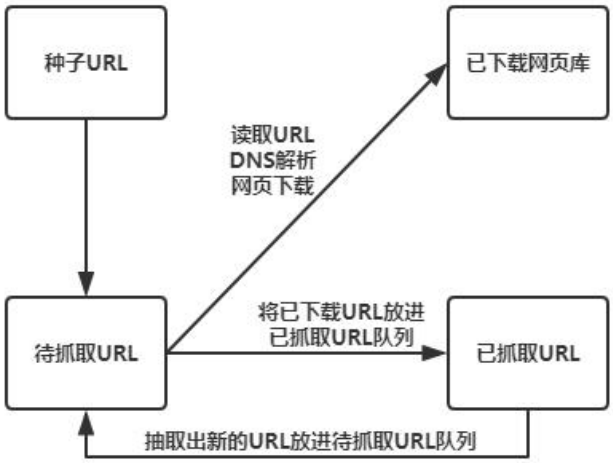
### 原理

网络爬虫，也叫网络蜘蛛（spider），是一种用来自动浏览万维网的网络机器人。其目的一般为编纂网络索引。网络搜索引擎等站点通过爬虫软件更新自身的网站内容或其对其他网站的索引。网络爬虫可以将自己所访问的页面保存下来，以便搜索引擎事后生成索引供用户搜索。

爬虫访问网站的过程会消耗目标系统资源。不少网络系统并不默许爬虫工作。因此在访问大量页面时，爬虫需要考虑到规划、负载，还需要讲“礼貌”。 不愿意被爬虫访问、被爬虫主人知晓的公开站点可以使用 robots.txt 文件之类的方法避免访问。这个文件可以要求机器人只对网站的一部分进行索引，或完全不作处理。

爬虫还可以验证超链接和 HTML 代码，用于网络抓取。

网络爬虫是搜索引擎抓取系统的重要组成部分。爬虫的主要目的是将互联网上的网页下载到本地形成一个互联网内容的镜像备份。




（通用的爬虫框架）

网络爬虫的基本工作流程如下：

(1)首先选取目标 URL；

(2)将目标 URL 放入待抓取 URL 队列；

(3)从待抓取 URL 队列中取出待抓取在 URL，解析 DNS，并且得到主机的 ip，并将 URL 对应的网页下载下来，存储进已下载网页库中。此外，将这些 URL 放进已抓取 URL 队列。

(4)分析已抓取 URL 队列中的 URL，分析其中的其他 URL，并且将 URL 放入待抓取 URL 队列，从而进入下一个循环。

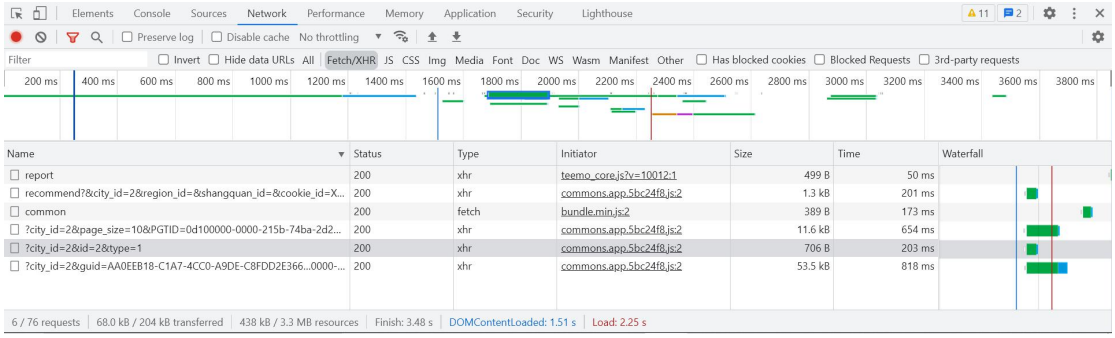
在爬虫系统中，待抓取 URL 队列是很重要的一部分。待抓取 URL 队列中的 URL 以什么样的顺序排列也是一个很重要的问题，因为这涉及到先抓取那个页面，后抓取哪个页面。而决定这些 URL 排列顺序的方法，叫做抓取策略。

# 操作步骤

数据采集部分使用 requsets 进行爬取，爬取网站为 58 同城，把获取的数据利用 pandas 保存到 csv 中。查看网站详细信息，得出 58 同城中的 URL 和他的请求参数，并得到他的返回为 JSON 串。

URL = 'https://sh.58.com/ershoufang/p{}'

找到存有我们信息的地址：




1. 爬取数据

首先，我们要确定，我们是使用哪一种的爬虫方式，由于我们在前期查找数据保存在什么地方的时候发现，我们需要的数据是在返回的 JSON 串里，因此我们不需要在网页中定位来获取我们的数据，只要获取返回的 json 串，在进行相关的处理，就获取我们需要的信息，因此我们采用了 python 的 requsets 库，来进行获取我们需要的结果。

数据爬取代码：

```
import time
from urllib import request
from urllib.parse import *

import requests
from bs4 import BeautifulSoup
import gzip
import json
from io import BytesIO
import string
import random
import os

headers = [
    "Mozilla/5.0 (Windows NT 6.1; Win64; rv:27.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.36"
    "Mozilla/5.0 (Windows NT 10.0; WOW64; rv:27.0) Gecko/20100101 Firfox/27.0"
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.36"
```

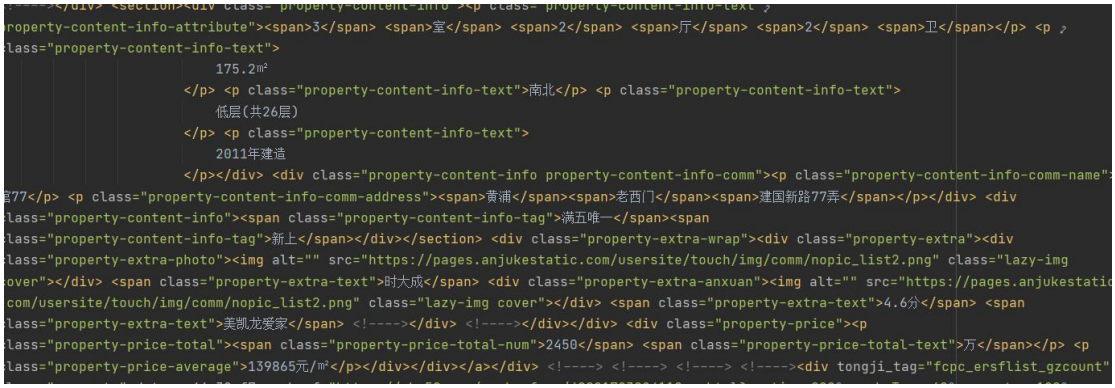
```
    "Mozilla/5.0 (Windows NT 10.0; WOW64; rv:10.0) Gecko/20100101 Firfox/10.0"
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/21.0.1180.110 Safari/537.36"
    "Mozilla/5.0 (X11; Ubuntu; Linux i686 rv:10.0) Gecko/20100101 Firfox/27.0"
    "Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/537.1 (KHTML, like Gecko) Chrome/34.0.1838.2 Safari/537.36"
    "Mozilla/5.0 (X11; Ubuntu; Linux i686 rv:27.0) Gecko/20100101 Firfox/27.0"
    "Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.36"
    'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/55.0.2883.87 Safari/537.36'
]


def get_content(url, headers, host):
    # @url：需要登录的网址
    # @headers：模拟的登陆的终端
    # ********************模拟登陆获取网址********************

    random_header = random.choice(headers)
    req = request.Request(url)
    req.add_header("User-Agent", random_header)
    req.add_header("Host", host)
    req.add_header("Accept",
"text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9")
    # req.add_header("Accept","*/*")
    req.add_header("Accept-Encoding", "gzip,deflate,br")
    req.add_header("Accept-Language", "zh-CN,zh;q=0.9")
    # req.add_header("Connection", "keep-alive")
    req.add_header("Cookie", 'f=n; commontopbar_new_city_info=2%7C%E4%B8%8A%E6%B5%B7%7Csh; userid360_xml=70ADB45668CA26707AE500336729E4D4; time_create=1651904061865; f=n; commontopbar_new_city_info=2%7C%E4%B8%8A%E6%B5%B7%7Csh; commontopbar_ipcity=xx%7C%E6%96%B0%E4%B9%A1%7C0; SECKEY_ABVK=trDSUUgc7IpE9v1ah5wWz8Q2mS6uCh4ZDYob94N7pxQ%3D; BMAP_SECKEY=XjThuhxn0KYn2uMszfPZSE2j37N7Ps2m28uYwjTVZfcMBDGZWTTw9xG6-OfgZWRuw4Ihf3trQf-fDe9Npgc_3obDryDHmpSTdjhJomQejd2mKA99L2dD5HGfC6qgrkOZp6BCE0w1Y7kf5Dmsz6NVXsRGy1p6u1Qn6bIowiiJFB6ypBFP10il6wrrPIQyPaLm; city=bj; id58=CocN02JOXMNAycxTKWC0Ag==; 58tj_uuid=c39662d8-c716-46b9-a8d7-0bc59e39b060; 58home=xx; myfeet_tooltip=end; wmda_new_uuid=1; wmda_uuid=413d1b5fe5b319133ad8079013a8047d; wmda_visited_projects=%3B11187958619315; als=0;
```

```
    xxzl_deviceid=l0v7S2iZ99PD5xEgy4ipXhepeLvoGzAGcXs4%2FM47vw4iOs5w%2BD9A30Hlnx
    ZziVCh; Hm_lvt_3f405f7f26b8855bc0fd96b1ae92db7e=1649302732,1649312053;
    aQQ_ajkguid=560804E1-F053-4081-ADE1-996B2C3A65BF;
    sessid=69044730-EFA2-4018-A00C-6B742090F518; ajk-appVersion=;
    fzq_h=41fe926928513e582e552d653b13cca6_1649312050308_5eae8956b6e047d18be5869
    abded9116_3659345452;
    init_refer=https%253A%252F%252Fsh.58.com%252Fershoufang%252Fp1%252F; new_uv=5;
    utm_source=; spm=; 58_ctid=2; is_58_pc=1;
    commontopbar_new_city_info=11%7C%E4%B8%8A%E6%B5%B7%7Csh; ctid=2;
    www58com="UserID=64268387810830&UserName=4g7a7be2n";
    58cooper="userid=64268387810830&username=4g7a7be2n"; 58uname=4g7a7be2n;
    passportAccount="atype=0&bstate=0"; new_session=0;
    PPU=UID=64268387810830&UN=4g7a7be2n&TT=616d5236b012978391695b015c517803&PBOD
    Y=T0vT9TFzKT53C3yEfFMT5zlE6JwqujyH1RmudZL0t-nfZDX4_Dp76IrIvViY18uFPNhJMi29eJ
    LQ0uIlyAGd5R2tNG82uFdtQiUq6z9j88pLc79wilBPtfPK6TrC5VdEToca4bgbo9vQflB8dSxhgE
    dRzGb-4cc-zB4uKofxYnQ&VER=1&CUID=E_osZYpwVo00_IqXsL8Rbw;
    xxzl_cid=1ebb0915e5e64a6eb87a939551c70600;
    xzuid=498044a4-7401-4799-a2f7-26fc6b8d3c01')
    try:
        response = request.urlopen(req)
        contents = response.read()
        if isinstance(contents, bytes):
            buff = BytesIO(contents)
            f = gzip.GzipFile(fileobj=buff)
            contents = f.read()
            contents = contents.decode('utf-8')
        else:
            print('service mysql restart')
        print('OK! ! ! ')
        return (contents)
    except Exception as e:
        print(e)


def get_links_from(page):
    # @page：表示第几页信息
    # @urls：所有列表的超链接，即子页网址
    # ****************此网站需要模拟登陆**********************
    # 返回全部子网页地址
    # https://search.51job.com/list/000000,000000,0000,00,9,99,+,2,11.html
    urls = []
    path = 'E:/zhaopin/2222/'
    if os.path.exists(path) == False:
        os.makedirs(path)
```

```
    for i in range(1, page):
        url = 'https://sh.58.com/ershoufang/p{}'.format(i)

        # url = quote(url, safe=string.printable)  data-v-28819723 data-v-d4f07c36
        info = get_content(url, headers, "sh.58.com")
        info = info.replace(" data-v-28819723","")
        info = info.replace(" data-v-d4f07c36", "")

        if info != None:
            with open(path + str(i) + ".html", 'w') as f:
                f.write(info)
        time.sleep(5)

if __name__ == '__main__':
    # get_recuite_info(2)
    get_links_from(100)
```

## 运行结果

爬取结果：




## 2.2 数据清洗

## 原理

数据清洗是对数据进行重新审查和校验的过程，目的在于删除重复信息、纠正存在的错误，并提供数据一致性。

数据清洗从名字上也看的出就是把“脏”的“洗掉”，指发现并纠正数据文件中可识别的错误的最后一道程序，包括检查数据一致性，处理无效值、缺失值等。因为数据仓库中的

数据是面向某一主题的数据的集合，这些数据从多个业务系统中抽取而来，而且包含历史数据，这样就避免不了有的数据是错误数据、有的数据相互之间有冲突，这些错误的或有冲突的数据显然是我们不想要的，称为“脏数据”。我们要按照一定的规则把“脏数据”“洗掉”，这就是数据清洗。而数据清洗的任务是过滤那些不符合要求的数据，将过滤的结果交给业务主管部门，确认是否过滤掉，还是由业务单位修正之后再进行抽取。不符合要求的数据主要是有不完整的数据、错误的数据、重复的数据三大类。数据清洗是与问卷审核不同，录入后的数据清洗一般是由计算机而不是人工完成。

### csv 数据文件的样式：

CSV：逗号分隔值（Comma-Separated Values，CSV，有时也称为字符分隔值，因为分隔字符也可以不是逗号），其文件以纯文本形式存储你表格数据（数字和文本）。文本意味着该文件是一个字符序列，不含必须像二进制数字那样被解读的数据。CSV 文件由任意数目的记录组成，记录间以某种换行符分隔；每条记录由字段组成，字段间的分隔符是其他字符或字符串，最常见的是逗号或制表符。通常搜索记录都有完全相同的字段序列。通常都是纯文本文件。

## 操作步骤

### （1）第一次清洗：

通过数据采集部分得出的数据显示，采集部分得到的数据还是比较干净的不需要过多的清洗，我们需要的就是保留我们后面所需要的指标的一些字段删除多余字段，这里我们后期分析以及可视化需要的字段有：

```
id  # 房屋标题
cengshu  # 房屋楼层
fangling  # 房龄
huxing  # 户型
jiage  # 房屋价格
mianji  # 房屋面积
pingfen  # 房屋评分
title  # 房屋户型
weizhi # 房屋地址
zhongjie # 房屋中介
zongjia # 房屋总价
```

其次我们需要将某些字段提取出所需要的关键信息，以便后面的可视化。

清洗不符合要求的字符，由于 csv 格式的文件是由逗号分开的，我们需要把一列中的逗号进行替换，除此之外，我们同时把其他符号也进行了替换，防止后续不必要的麻烦。我们通过循环每一列，对每一列的符号替换成空格。在保存到新的 csv 中。

```
1. from bs4 import BeautifulSoup
2. import pandas as pd
```

```
import os
import re


def get_html_info(html):
    '''''
    *****************获取网页的有用信息并保存成字典形式****************
    '''
    try:
        datalist = []
        soup = BeautifulSoup(html, "lxml")  # 设置解析器为“lxml”
        content_list = soup.select("section.list")[0]
        contents = content_list.select("div.property-content")
        for content in contents:

            huxing = content.select("div.property-content-info")[0].select("p")[0]
            hx1=huxing.text.strip().replace(' ','').split('室')[0]
            hx2 = huxing.text.strip().replace(' ','').split('室')[1].split('厅')[0]
            hx3 = huxing.text.strip().replace(' ','').split('室')[1].split('厅')[1][0:1]
            mianji = content.select("div.property-content-info")[0].select("p")[1]
            print(mianji.text.strip())
            cengshu = content.select("div.property-content-info")[0].select("p")[3]
            print(cengshu.text.strip().split('(')[0][0:1])
            if cengshu.text.strip().split('(')[0][0:1]=='共':
                cengshu = '其他'
            else:
                cengshu=cengshu.text.strip().split('(')[0]
            print(cengshu)
            fangling = content.select("div.property-content-info")[0].select("p")[4]
            print(fangling.text.strip())
            weizhi = content.select("div.property-content-info-comm p")[1]
            print(weizhi.text.strip())
            zongjia = content.select("div.property-price p")[0]
            print(zongjia.text.strip())


            data = {
                "price": zongjia.text.strip().replace(" 万", ""),
```

```
                "room": hx1,
                "living": hx2,
                "rest": hx3,
                "area": mianji.text.strip().replace("㎡",""),
                "floor": cengshu,
                "age": fangling.text.strip().replace("年建造",""),
                "location": weizhi.text.strip()[0:2]
            }
            datalist.append(data)
        return (datalist)
    except Exception:
        pass

    # 将全部信息保存成DataFrame形式，去除无用信息


def get_htmls_all_info(path):
    df = pd.DataFrame({
        "price": [],
        "room": [],
        "living": [],
        "rest": [],
        "area": [],
        "floor": [],
        "age": [],
        "location": []
    })
    dirs = os.listdir(path)
    for dir in dirs:
        # print(dir)
        if dir.find('swp'):
            dir = dir.strip('.' and '.swp')
        p = os.path.join(path, dir)
        html = open(p, encoding='gbk').read()
        data = get_html_info(html)
        print(data)
        df = df.append(data, ignore_index=True)
    return df


if __name__ == '__main__':
    path = 'D:/zhaopin/2222'
    df = get_htmls_all_info(path)
    df.to_csv('D:/zhaopin/bigdata4', index=False,sep='\t')
```

```
    print(df)
```

## （2）第二次清洗

下面进行第二次数据清洗，我们需要将某些字段提取出所需要的关键信息，以便后面的分析。

```
from bs4 import BeautifulSoup
import pandas as pd
import os
import re


def get_html_info(html):
    '''''
    *****************获取网页的有用信息并保存成字典形式****************
    '''
    try:
        datalist = []
        soup = BeautifulSoup(html, "lxml")  # 设置解析器为“lxml”
        content_list = soup.select("section.list")[0]
         contents = content_list.select("div.property-content")
         for content in contents:

            huxing = content.select("div.property-content-info")[0].select("p")[0]
            hx1=huxing.text.strip().replace(' ','').split('室')[0]
            hx2 = huxing.text.strip().replace(' ','').split('室')[1].split('厅')[0]
            hx3 = huxing.text.strip().replace(' ','').split('室')[1].split('厅')[1][0:1]
            mianji = content.select("div.property-content-info")[0].select("p")[1]
            print(mianji.text.strip())
            cengshu = content.select("div.property-content-info")[0].select("p")[3]
            print(cengshu.text.strip().split('(')[0][0:1])
            if cengshu.text.strip().split('(')[0][0:1]=='共':
                cengshu = '其他'
            else:
                cengshu=cengshu.text.strip().split('(')[0]
            print(cengshu)
            fangling = content.select("div.property-content-info")[0].select("p")[4]
            print(fangling.text.strip())
            weizhi = content.select("div.property-content-info-comm p")[1]
```

```
            print(weizhi.text.strip())
            zongjia = content.select("div.property-price p")[0]
            print(zongjia.text.strip())


            data = {
                "price": zongjia.text.strip().replace(" 万", ""),
                "room": hx1,
                "living": hx2,
                "rest": hx3,
                "area": mianji.text.strip().replace("㎡",""),
                "floor": cengshu,
                "age": fangling.text.strip().replace("年建造",""),
                "location": weizhi.text.strip()[0:2]
            }
            datalist.append(data)
        return (datalist)
    except Exception:
        pass

    # 将全部信息保存成 DataFrame 形式，去除无用信息


def get_htmls_all_info(path):
    df = pd.DataFrame({
        "price": [],
        "room": [],
        "living": [],
        "rest": [],
        "area": [],
        "floor": [],
        "age": [],
        "location": []
    })
    dirs = os.listdir(path)
    for dir in dirs:
        # print(dir)
        if dir.find('swp'):
            dir = dir.strip('.' and '.swp')
        p = os.path.join(path, dir)
        html = open(p, encoding='gbk').read()
        data = get_html_info(html)
        print(data)
        df = df.append(data, ignore_index=True)
```

```
    return df


if __name__ == '__main__':
    path = 'D:/zhaopin/2222'
    df = get_htmls_all_info(path)
    df.to_csv('D:/zhaopin/bigdata4', index=False,sep='\t')
    print(df)
```

## 运行结果

第一次数据清洗结果：

对象 house @house (192.168.205...

开始事务 备注 筛选 排序 导入 导出

| id | cengshu | fangling | huxing | jiage | mianji | pingfen | title | weizhi | zhongjie | zongjia |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 高层(共6层) | 1997年建造 | 2 室 1 厅 1 | 54857 | 89 | 3.6 | 保德公寓 | 静安彭浦保 | 黄丹丹 | 488 |
| 1 | 低层(共21层) | 1997年建造 | 2 室 2 厅 1 | 102504 | 92.7 | 3.9 | 芝大厦（ | 徐汇衡山路 | 杨旭 | 950 |
| 2 | 低层(共7层) | 1996年建造 | 2 室 1 厅 1 | 71420 | 76.9 | 3.9 | 浣纱六村 | 杨浦黄兴浣 | 陈德华 | 549 |
| 3 | 共5层 | 1999年建造 | 3 室 2 厅 1 | 47903 | 93.9 | 3.5 | 景观房， | 浦东康桥康 | 李洁 | 450 |
| 4 | 中层(共13层) | 2010年建造 | 2 室 2 厅 1 | 44490 | 78.2 | 4.7 | 今日 可 | 松江松江大 | 刘飞 | 348 |
| 5 | 高层(共17层) | 2012年建造 | 2 室 2 厅 2 | 26316 | 76 | 4.6 | 11号地铁 | 嘉定南翔丰 | 吕喜燕 | 200 |
| 6 | 中层(共7层) | 1997年建造 | 2 室 1 厅 1 | 79596 | 86.1 | 4.5 | 环境优美 | 长宁天山威 | 张旭伟 | 685 |
| 7 | 高层(共18层) | 2010年建造 | 2 室 2 厅 1 | 34362 | 74.2 | 4.8 | 商zhu两 | 静安西藏北 | 袁茜 | 255 |
| 8 | 高层(共24层) | 2008年建造 | 3 室 2 厅 1 | 51500 | 100 | 4.9 | 普陀长风 | 普陀长风丹 | 毛恩迪 | 515 |
| 9 | 中层(共12层) | 2014年建造 | 2 室 1 厅 1 | 55925 | 76.9 | 3.5 | 近地铁 繁 | 浦东周浦川 | 张峰 | 430 |
| 10 | 中层(共6层) | 2006年建造 | 2 室 1 厅 1 | 13115 | 61 | 2.4 | 精装修 6 | 奉贤奉城天 | 王江 | 80 |
| 11 | 低层(共6层) | 1996年建造 | 2 室 1 厅 1 | 68074 | 58.8 | 4.6 | 打一！带 | 杨浦新华医 | 孙红 | 400 |
| 12 | 低层(共6层) | 1995年建造 | 2 室 1 厅 1 | 34770 | 87.7 | 4.3 | 地铁18号 | 浦东周浦浦 | 孙光辉 | 305 |
| 13 | 中层(共26层) | 2012年建造 | 2 室 2 厅 1 | 28750 | 80 | 4.8 | 东外滩豪 | 杨浦杨浦周 | 楚亚涛 | 230 |
| 14 | 低层(共26层) | 2011年建造 | 3 室 2 厅 2 | 139865 | 175.2 | 4.6 | 新天地 次 | 黄浦老西门 | 时大成 | 2450 |
| 15 | 中层(共19层) | 2012年建造 | 2 室 1 厅 1 | 43750 | 80 | 4.0 | （无社保 | 静安静安寺 | 陈恒 | 350 |
| 16 | 高层(共17层) | 2008年建造 | 4 室 1 厅 2 | 49223 | 128 | 4.7 | 电梯房 新 | 闵行莘庄莘 | 刘大宝 | 630 |
| 17 | 高层(共24层) | 2016年建造 | 3 室 2 厅 2 | 79161 | 126.2 | 4.8 | 据一套首 | 杨浦平凉路 | 张智杰 | 999 |
| 18 | 中层(共26层) | 2000年建造 | 3 室 2 厅 2 | 79052 | 139.2 | 4.6 | 康桥 3室 | 浦东康桥秀 | 马盼盼 | 1100 |
| 19 | 低层(共22层) | 2012年建造 | 3 室 2 厅 1 | 56123 | 98 | 4.6 | 静安不夜 | 静安新客站 | 吕艳荣 | 550 |
| 20 | 低层(共18层) | 2015年建造 | 3 室 1 厅 1 | 31590 | 82.9 | 4.6 | 近罗南新 | 宝山罗南美 | 赵美奇 | 262 |
| 21 | 高层(共18层) | 2015年建造 | 1 室 1 厅 1 | 28645 | 59 | 4.8 | 公寓房 民 | 普陀真如华 | 袁茜 | 169 |
| 22 | 共5层 | 2019年建造 | 2 室 2 厅 1 | 10484 | 62 | 4.5 | 外地人不 | 松江叶榭官 | 杜志超 | 65 |
| 23 | 高层(共32层) | 2008年建造 | 4 室 2 厅 3 | 97217 | 226.3 | 5.0 | 坐拥陆家 | 虹口北外滩 | 江文 | 2200 |
| 24 | 低层(共20层) | 2015年建造 | 2 室 2 厅 1 | 36045 | 88.8 | 4.8 | 诚售，丝 | 浦东新场申 | 秦卫荣 | 320 |
| 25 | 中层(共18层) | 2015年建造 | 2 室 2 厅 2 | 45000 | 60 | 4.6 | 城市标杆 | 长宁天山天 | 李笑笑 | 270 |

第二次数据清洗结果：

```
price room living rest area floor age location
380 3 2 2 93.2 中层 1996 宝山
292 3 2 1 88.7 低层 2012 浦东
342 2 1 1 91.1 高层 2006 浦东
415 2 2 1 81.1 高层 2004 浦东
410 2 2 1 94.5 中层 2007 浦东
430 2 2 1 86 中层 2000 普陀
308 2 2 1 80 低层 2015 青浦
358 2 2 2 97 中层 2010 闵行
390 2 2 1 94.1 低层 1997 长宁
400 2 2 1 91 中层 2004 浦东
275 2 1 1 91.8 中层 2005 嘉定
308 2 2 1 82 中层 2000 松江
295 2 2 1 88.1 中层 2016 浦东
358 2 1 1 89.1 高层 1996 闵行
342 2 2 1 81.5 低层 2012 嘉定
400 2 1 1 91 高层 1997 宝山
316 2 2 1 86.3 低层 2009 松江
300 2 1 1 81.2 高层 2013 浦东
378 2 2 1 91.2 中层 2008 浦东
390 2 2 1 91 高层 1995 浦东
428 2 2 2 97 中层 2009 浦东
415 2 2 1 82 低层 2000 浦东
330 2 1 1 80.4 中层 1995 闵行
300 2 2 1 92.9 低层 2012 浦东
429.5 2 2 1 89.2 高层 2000 浦东
328 2 2 1 91.1 中层 2016 青浦
415 2 2 1 90.1 中层 2004 宝山
```

## 2.3 数据分析

### 原理

- Spark 数据处理方式主要有三种：RDD、DataFrame、Spark SQL

  三者的主要差异在于是否定义 Schema。

  RDD 的数据未定义 Schema(也就是未定义字段名及数据类型)。使用上必须有 Map/Reduce 的概念，需要高级别的程序设计能力。但是功能也最强，能完成所有 Spark 功能。

  Spark DataFrame 建立时必须定义 Schema（定义每一个字段名与数据类型）

  Spark SQL 是由 DataFrame 衍生出来的，我们必须先建立 DataFrame，然后通过登录 Spark SQL temp table，就可以使用 Spark SQL 语法了。

  易使用度：Spark SQL>DataFrame>RDD，DataFrame 与 SparkSQL 比 RDD 更快速。

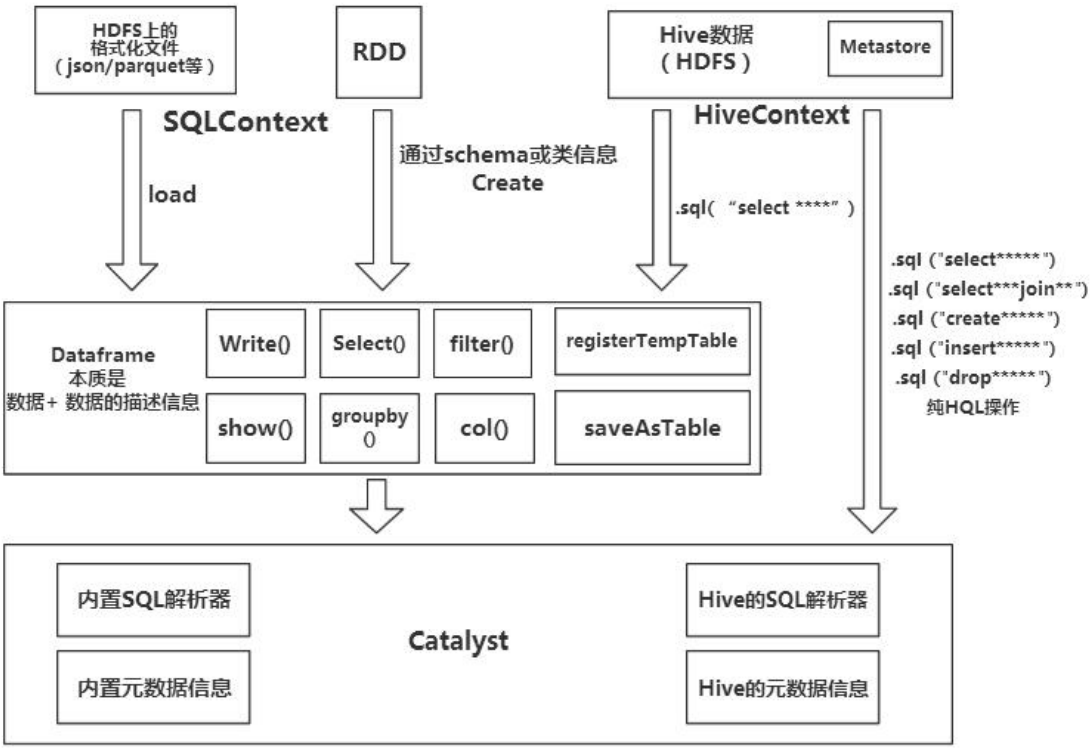
  DataFrame 与 Spark SQL 通过 Catalyst 进行最优化，可以大幅提高执行效率。Python 语言特性使其使用 RDD 时执行速度比 Scala 慢，但是 Spark Python 使用 DataFrame 时，执行性能几乎与 Spark Scala 使用 DataFrame 相同，而且使用 DataFrames 时，无论是 Python 还是 Scala，运行时间都明显比使用 RDD 少很多。

- Spark SQL 重要的是操作 DataFrame，DataFrame 本身提供了 Save 和 Load 的操作，

  Load：可以创建 DataFrame。

  Save：把 DataFrame 中的数据保存到文件或者说用具体的格式来指明我们要读取的文件类型，以及用具体的格式来指出我们要输出的文件是什么类型。

  Spark SQL 执行基本操作时，内部结构流程图如下：




  DataFrame 本质是数据 + 数据的描述信息（结构元信息）。

  所有的上述 SQL 及 DataFrame 操作最终都通过 Catalyst 翻译成 Spark 程序 RDD 操作代码。

- SQLContext 使用：

  1. 简单的解析器（Scala 语言写的 SQL 解析器）比如：

  （1）在半结构化的文件里面使用 SQL 查询时，是用这个解析器解析的。

  （2）访问（半）结构化文件的时候，通过 sqlContext 使用 schema，类生成 DataFrame，然后 DataFrame 注册为表时，registerTempTable（注册临时表）然后从这个表里面进行查询时，即使用的简单的解析器。

  simpleCatalog 此对象中存放关系（表），比如我们指定的 schema 信息，类的信息，都是关系信息。

  2. HiveContext 使用：

  （1）HiveQL 解析器支持 Hive 的 HQL 语法，如只有通过 HiveContext 生成的 DataFrame 才能调用 saveAsTable 操作。

  （2）hiveCatalog（存放数据库和表的元数据信息）

特点

支持三种分词模式：

a.精确模式，试图将句子最精确地切开，适合文本分析；

b.全模式，把句子中所有的可以成词的词语都扫描出来，速度非常快，但是不能解决歧义；

c.搜索引擎模式，在精确模式的基础上，对长词再次切分，提高召回率，适合用于搜索引擎分词。

支持繁体分词

支持自定义词典

MIT 授权协议

Part 1. 分词

jieba.cut 方法接受三个输入参数: 需要分词的字符串；cut_all 参数用来控制是否采用全模式；HMM 参数用来控制是否使用 HMM 模型。

jieba.cut_for_search 方法接受两个参数：需要分词的字符串；是否使用 HMM 模型。该方法适合用于搜索引擎构建倒排索引的分词，粒度比较细。

待分词的字符串可以是 unicode 或 UTF-8 字符串、GBK 字符串。注意：不建议直接输入 GBK 字符串，可能无法预料地错误解码成 UTF-8。

jieba.cut 以及 jieba.cut_for_search 返回的结构都是一个可迭代的 generator，可以使用 for 循环来获得分词后得到的每一个词语(unicode)，或者用

jieba.lcut 以及 jieba.lcut_for_search 直接返回 list。

jieba.Tokenizer(dictionary=DEFAULT_DICT) 新建自定义分词器，可用于同时使用不同词典。jieba.dt 为默认分词器，所有全局分词相关函数都是该分词器的映射。

Part 2. 添加自定义词典

载入词典

开发者可以指定自己自定义的词典，以便包含 jieba 词库里没有的词。虽然 jieba 有新词识别能力，但是自行添加新词可以保证更高的正确率。

用法： jieba.load_userdict(file_name) # file_name 为自定义词典的路径。 词典格式和 dict.txt 一样，一个词占一行；每一行分三部分，一部分为词语，另一部分为词频（可省略），最后为词性（可省略），用空格隔开。

词频可省略，使用计算出的能保证分出该词的词频。

更改分词器的 tmp_dir 和 cache_file 属性，可指定缓存文件位置，用于受限的文件系统。

## 操作步骤

1. 在 IPython Notebook 中使用命令读取 HDFS 上的 house_data.csv 文件并查看文件内数据行数。

```
RDD1 = sc.textFile("hdfs://localhost:9000/house_data.csv")
```

2. 去掉第一行。

```
header = RDD1.first()
RDD1 = RDD1.filter(lambda row:row != header)
```

3. 使用 map 处理每一项数据，用 lambda 表达式创建匿名函数传入 line 参数。在匿名函数中，使用 line.split(",") 按照“,”为分隔符获取每一个字段。

```
RDD2=RDD1.map(lambda line:line.split(","))
```

4. 通过 RDD1 创建 DataFrame。导入 row 模块，定义 DataFrames 的每一个字段名与数据类型。

```
house_Rows = RDD2.map(lambda p:
Row(
companyFullName=p[0],
companyShortName=p[1],
companySize=p[2],
financeStage=p[3],
district=p[4],
positionName=p[5],
workyear=p[6],
education=p[7],
salary=p[8],
positionWelfare=p[9],
businessbRange=p[10],
firstType=p[11],
companyLabelList=p[12],
secondType=p[13],
city=p[14],
mean=p[15],
grade=p[16],
province=p[17]
)
)
```

5. 创建了 house_Rows 之后，使用 sqlContext.createDataFrame()方法传入 house_Rows 数据，创建 DataFrame，并改名为”df”。

```
house_df = sqlContext.createDataFrame(house_Rows)
house_df.alias("df")
```

创建 Spark SQL

6. 使用 registerDataFrameAsTable 方法将 df 转换为 house_table 表。

```
sqlContext.registerDataFrameAsTable(df, "house_table")
```

使用 sqlContext.sql()输入 sql 语句，使用 select 关键字查询所需字段的数据，并使用 from 关键字指定要查询的表，最后使用 show()方法显示查询结果。

7. 分析二手房的面积情况、价格、二手房房量、总价、房龄情况

```
from bs4 import BeautifulSoup
import pandas as pd
import os
import re


def get_html_info(html):
    '''''
    *****************获取网页的有用信息并保存成字典形式****************
    '''
    try:
        datalist = []
        soup = BeautifulSoup(html, "lxml")  # 设置解析器为“lxml”
        content_list = soup.select("section.list")[0]
        contents = content_list.select("div.property-content")
        for content in contents:

            huxing = content.select("div.property-content-info")[0].select("p")[0]
            hx1=huxing.text.strip().replace(' ','').split('室')[0]
            hx2 = huxing.text.strip().replace(' ','').split('室')[1].split('厅')[0]
            hx3 = huxing.text.strip().replace(' ','').split('室')[1].split('厅')[1][0:1]
            mianji = content.select("div.property-content-info")[0].select("p")[1]
            print(mianji.text.strip())
```

```
            cengshu = content.select("div.property-content-info")[0].select("p")[3]
            print(cengshu.text.strip().split('(')[0][0:1])
            if cengshu.text.strip().split('(')[0][0:1]=='共':
                cengshu = '其他'
            else:
                cengshu=cengshu.text.strip().split('(')[0]
            print(cengshu)
            fangling = content.select("div.property-content-info")[0].select("p")[4]
            print(fangling.text.strip())
            weizhi = content.select("div.property-content-info-comm p")[1]
            print(weizhi.text.strip())
            zongjia = content.select("div.property-price p")[0]
            print(zongjia.text.strip())


            data = {
                "price": zongjia.text.strip().replace(" 万", ""),
                "room": hx1,
                "living": hx2,
                "rest": hx3,
                "area": mianji.text.strip().replace("㎡",""),
                "floor": cengshu,
                "age": fangling.text.strip().replace("年建造",""),
                "location": weizhi.text.strip()[0:2]
            }
            datalist.append(data)
        return (datalist)
    except Exception:
        pass

    # 将全部信息保存成 DataFrame 形式，去除无用信息


def get_htmls_all_info(path):
    df = pd.DataFrame({
        "price": [],#总价
        "room": [],
        "living": [],
        "rest": [],
        "area": [],#面积
        "floor": [],#楼层
        "age": [],#房龄
```

```
66.  “zongjie” []: #评价
67. "location": []
68.    })
69.    dirs = os.listdir(path)
70.    for dir in dirs:
71.        # print(dir)
72.        if dir.find('swp'):
73.            dir = dir.strip('.' and '.swp')
74.        p = os.path.join(path, dir)
75.        html = open(p, encoding='gbk').read()
76.        data = get_html_info(html)
77.        print(data)
78.        df = df.append(data, ignore_index=True)
79.    return df
80.
81.
82.if __name__ == '__main__':
83.    path = 'D:/zhaopin/2222'
84.    df = get_htmls_all_info(path)
85.    df.to_csv('D:/zhaopin/bigdata4', index=False,sep='\t')
86.    print(df)
```

## 运行结果

```
price   room    living  rest    area    floor   age location
380 3   2   2   93.2    中层    1996    宝山
292 3   2   1   88.7    低层    2012    浦东
342 2   1   1   91.1    高层    2006    浦东
415 2   2   1   81.1    高层    2004    浦东
410 2   2   1   94.5    中层    2007    浦东
430 2   2   1   86  中层    2000    普陀
308 2   2   1   80  低层    2015    青浦
358 2   2   2   97  中层    2010    闵行
390 2   2   1   94.1    低层    1997    长宁
400 2   2   1   91  中层    2004    浦东
275 2   1   1   91.8    中层    2005    嘉定
308 2   2   1   82  中层    2000    松江
295 2   2   1   88.1    中层    2016    浦东
358 2   1   1   89.1    高层    1996    闵行
342 2   2   1   81.5    低层    2012    嘉定
400 2   1   1   91  高层    1997    宝山
316 2   2   1   86.3    低层    2009    松江
300 2   1   1   81.2    高层    2013    浦东
378 2   2   1   91.2    中层    2008    浦东
390 2   2   1   91  高层    1995    浦东
428 2   2   2   97  中层    2009    浦东
415 2   2   1   82  低层    2000    浦东
330 2   1   1   80.4    中层    1995    闵行
300 2   2   1   92.9    低层    2012    浦东
429.5   2   2   1   89.2    高层    2000    浦东
328 2   2   1   91.1    中层    2016    青浦
415 2   2   1   90.1    中层    2004    宝山
358 2   1   1   81.4    中层    1998    闵行
```

开始事务 备注 筛选 排序 导入 导出

| id | zongjia | num |
|---|---|---|
| 0 | 300 - 350 | 1583 |
| 1 | 500 - 800 | 1564 |
| 2 | 350 - 400 | 1278 |
| 3 | 400 - 500 | 1181 |
| 4 | 1000以上 | 1081 |
| 5 | 250 - 300 | 954 |
| 6 | 100 - 200 | 595 |
| 7 | 100以下 | 557 |
| 8 | 800 - 1000 | 484 |
| 9 | 200 - 250 | 263 |

1000w 以上的房子大概占到 10%左右，若 300-1000w，由此我们可以得到中等价格二手房最多，其次是低价的二手房，高价二手房最少，由此可以得出去上海买二手房的一般都会选择中等价格的二手房。

| id | mianji | num |
|---|---|---|
| 0 | 80 - 90 | 2629 |
| 1 | 90 - 100 | 1920 |
| 2 | 120 - 140 | 1888 |
| 3 | 60 - 70 | 844 |
| 4 | 70 - 80 | 816 |
| 5 | 100 - 120 | 665 |
| 6 | 50 - 60 | 526 |
| 7 | 50以下 | 252 |

分析：普及一个知识，90 平米以下的户型为小户型，90~144 平米之间的户型称之为中户型，144 平米以上的户型称之为大户型

大概来看，小户型大概占到总二手房的 40%左右，中等户型 90-140 占到 60%左右

我们可以得出结论：一般去上海买二手房的人都会选择中等户型的房子，可能因为有了家庭，小户型不够住，大户型买不起。

| id | fangling | num |
|---|---|---|
| 0 | 2010年以后 | 1155 |
| 1 | 2000 - 2010年 | 1151 |
| 2 | 1990 - 2000 | 325 |
| 3 | 1970 - 1990 | 8 |
| 4 | 1970年以前 | 1 |

分析：从表中我们可以看出，2010 年以后与 2000-2010 年的二手房人几乎持平，占二手房的大多数，而 1990 到 2000 年的占 10%左右，1990 年前几乎没有二手房，根据此，可以得出结论：上海在 1990 年前二手房产业几乎还没有开始，到达 1990 年开始初步发展，到达 2000 年开始蓬勃发展。

开始事务 备注 筛选 排序 导入 导出

| id | cengshu | num |
|---|---|---|
| 0 | 低层 | 1173 |
| 1 | 中层 | 899 |
| 2 | 高层 | 375 |
| 3 | 其他 | 193 |

分析结果：排除替他情况外

发现低层二手房占总二手房的大多数并且可以得到二手房房量于层数的高低成反比

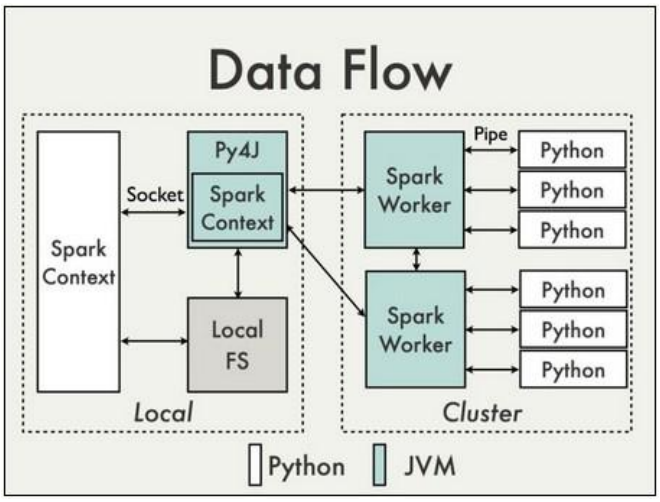
即楼层越高，二手房就越少，进而可以推出，可以得出上海底层的房子不太受欢迎，楼层越高越受欢迎.

## 2.4 数据导出迁徙

### 原理

PySpark 是 Spark 为 Python 开发者提供的 API ，位于 $SPARK_HOME/bin 目录，其依赖于 Py4J。

大体用下面这张图来表示 PySpark 的实现机制：




在 python driver 端,SparkContext 利用 Py4J 启动一个 JVM 并产生一个 JavaSparkContext。Py4J 只使用在 driver 端，用于本地 python 与 java SparkContext objects 的通信。大量数据的传输使用的是另一个机制。

RDD 在 python 下的转换会被映射成 java 环境下 PythonRDD。在远端 worker 机器上,PythonRDD 对象启动一些子进程并通过 pipes 与这些子进程通信，以此 send 用户代码和数据。

### 操作步骤

1. 导入所需的库

```
import pandas as pd
from pyspark.sql import SQLContext
from pyspark import SparkContext,SparkConf
from pyspark.sql import Row
import pymysql
from sqlalchemy import create_engine
import os
```

2. 设置系统环境变量

```
os.environ['PYSPARK_PYTHON']='/usr/bin/python3.5'
```

3. 创建一个 SparkConf 对象。Spark 应用程序的配置，用于将各种 Spark 参数设置为键值对。

```
conf=SparkConf().setMaster("local[*]").setAppName("Fenxi").set("spark.cores.max",10)
```

4. 基于 SparkConf 创建一个 SparkContext 对象。驱动器程序通过 SparkContext 对象来访问 Spark。

```
sc=SparkContext(conf=conf)
```

5. SQLContext 是 Spark SQL 进行结构化数据处理的入口，可以通过它进行 DataFrame 的创建及 SQL 的执行。

```
sqlContext = SQLContext(sc)
```

在数据分析的基础上，将分析后的数据导入数据库中。

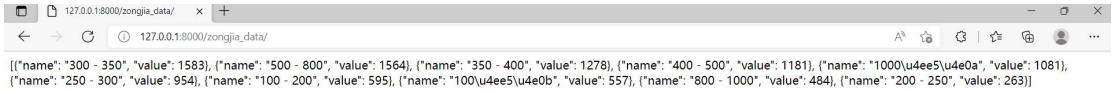
6. 分析二手房信息并导出到 mysql 中的 house 数据库

6.1 分析二手房的总价情况并导出到 mysql 中的 house 数据库

```
SELECT
 a.zongjia_qujian AS zongjia,
 count(*) AS num
FROM
 (
     SELECT
         CASE
     WHEN zongjia < 100 THEN
         '100 以下'
     WHEN zongjia < 200 THEN
         '100 - 200'
     WHEN zongjia < 250 THEN
         '200 - 250'
     WHEN zongjia < 300 THEN
         '250 - 300'
     WHEN zongjia < 350 THEN
         '300 - 350'
     WHEN zongjia < 400 THEN
         '350 - 400'
     WHEN zongjia < 500 THEN
         '400 - 500'
     WHEN zongjia < 800 THEN
         '500 - 800'
     WHEN zongjia < 1000 THEN
```

```
            '800 - 1000'
        ELSE
            '1000 以上'
        END AS zongjia_qujian
        FROM
            house_table
    ) AS a
GROUP BY
    a.zongjia_qujian
ORDER BY
    count(*) DESC
""").toPandas()
pd.io.sql.to_sql(
    df_mianji,
    name='mianji',
    con=conn,
    schema='house',
    if_exists='replace',
    index=True,
    index_label='id'
)

```

运行结果：




[{"name": "300 - 350", "value": 1583}, {"name": "500 - 800", "value": 1564}, {"name": "350 - 400", "value": 1278}, {"name": "400 - 500", "value": 1181}, {"name": "1000\u4ee5\u4e0a", "value": 1081}, {"name": "250 - 300", "value": 954}, {"name": "100 - 200", "value": 595}, {"name": "100\u4ee5\u4e0b", "value": 557}, {"name": "800 - 1000", "value": 484}, {"name": "200 - 250", "value": 263}]

开始事务 备注 筛选 排序 导入 导出

| id | zongjia | num |
|---|---|---|
| 0 | 300 - 350 | 1583 |
| 1 | 500 - 800 | 1564 |
| 2 | 350 - 400 | 1278 |
| 3 | 400 - 500 | 1181 |
| 4 | 1000以上 | 1081 |
| 5 | 250 - 300 | 954 |
| 6 | 100 - 200 | 595 |
| 7 | 100以下 | 557 |
| 8 | 800 - 1000 | 484 |
| 9 | 200 - 250 | 263 |

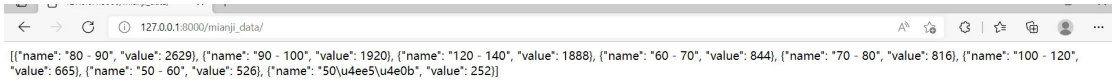
6.2 分析二手房的面积情况并导出到 mysql 中的 house 数据库

```
SELECT
```

```
  b.mianji_qujian AS mianji,
  count(*) AS num
 FROM
  (
      SELECT
          CASE
      WHEN mianji < 50 THEN
          '50 以下'
      WHEN mianji < 60 THEN
          '50 - 60'
      WHEN mianji < 70 THEN
          '60 - 70'
      WHEN mianji < 80 THEN
          '70 - 80'
      WHEN mianji < 90 THEN
          '80 - 90'
      WHEN mianji < 100 THEN
          '90 - 100'
      WHEN mianji < 120 THEN
          '100 - 120'
      WHEN mianji < 1400 THEN
          '120 - 140'
      WHEN mianji < 210 THEN
          '140 - 210'
      ELSE
          '210 以上'
      END AS mianji_qujian
      FROM
          house_table
  ) AS b
GROUP BY
  b.mianji_qujian
ORDER BY
  count(*) DESC
""").toPandas()

pd.io.sql.to_sql(
    df_mianji,
    name='mianji',
    con=conn,
    schema='house',
    if_exists='replace',
    index=True,
    index_label='id'
```

```
)

```

运行结果：




[{"name": "80 - 90", "value": 2629}, {"name": "90 - 100", "value": 1920}, {"name": "120 - 140", "value": 1888}, {"name": "60 - 70", "value": 844}, {"name": "70 - 80", "value": 816}, {"name": "100 - 120", "value": 665}, {"name": "50 - 60", "value": 526}, {"name": "50\u4ee5\u4e0b", "value": 252}]

| id | mianji | num |
|---|---|---|
| 0 | 80 - 90 | 2629 |
| 1 | 90 - 100 | 1920 |
| 2 | 120 - 140 | 1888 |
| 3 | 60 - 70 | 844 |
| 4 | 70 - 80 | 816 |
| 5 | 100 - 120 | 665 |
| 6 | 50 - 60 | 526 |
| 7 | 50以下 | 252 |

6.3 分析二手房的价格情况并导出到 mysql 中的 house 数据库

```
SELECT
 d.jiage_qujian AS jiage,
 count(*) AS num
FROM
 (
     SELECT
         CASE
     WHEN jiage < 15000 THEN
         '15000 以下'
     WHEN jiage < 30000 THEN
         '15000 - 30000'
     WHEN jiage < 50000 THEN
         '30000 - 50000'
     WHEN jiage < 80000 THEN
         '50000 - 80000'
     WHEN jiage < 100000 THEN
         '80000 - 100000'
     WHEN jiage < 130000 THEN
         '100000 - 130000'
     WHEN jiage < 150000 THEN
```

```
            '130000 - 150000'
        WHEN jiage < 180000 THEN
            '150000 - 180000'
        WHEN jiage < 200000 THEN
            '180000 - 200000'
        ELSE
            '200000 以上'
        END AS jiage_qujian
        FROM
            house_table
    ) AS d
GROUP BY
    d.jiage_qujian
ORDER BY
    count(*) DESC
""").toPandas()

pd.io.sql.to_sql(
    df_jiage,
    name='jiage',
    con=conn,
    schema='house',
    if_exists='replace',
    index=True,
    index_label='id'
)

```

运行结果：

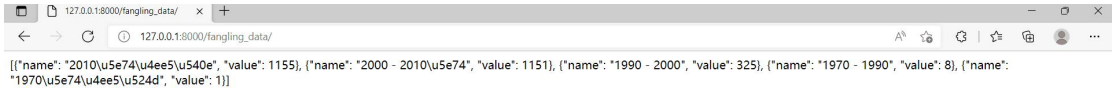
| id | jiage | num |
|---|---|---|
| 0 | 30000 - 5 | 2413 |
| 1 | 50000 - 8 | 133 |
| 2 | 15000 - 3 | 92 |
| 3 | 80000 - 1 | 2 |

6.4 分析二手房的房龄情况并导出到 mysql 中的 house 数据库

```
SELECT
```

```
  a.fangling_qujian AS fangling,
  count(*) AS num
FROM
  (
      SELECT
          CASE
      WHEN fangling < 1970 THEN
          '1970年以前'
      WHEN fangling < 1990 THEN
          '1970 - 1990'
      WHEN fangling < 2000 THEN
          '1990 - 2000'
      WHEN fangling < 2010 THEN
          '2000 - 2010年'
      ELSE
          '2010年以后'
      END AS fangling_qujian
      FROM
          house_table
  ) AS a
GROUP BY
  a.fangling_qujian
ORDER BY
  count(*) DESC
""").toPandas()

pd.io.sql.to_sql(
    df_fangling,
    name='fangling',
    con=conn,
    schema='house',
    if_exists='replace',
    index=True,
    index_label='id'
)
```

运行结果：




[{"name": "2010\u5e74\u4ee5\u540e", "value": 1155}, {"name": "2000 - 2010\u5e74", "value": 1151}, {"name": "1990 - 2000", "value": 325}, {"name": "1970 - 1990", "value": 8}, {"name": "1970\u5e74\u4ee5\u524d", "value": 1}]

| id | fangling | num |
|---|---|---|
| 0 | 2010年以后 | 1155 |
| 1 | 2000 - 2010年 | 1151 |
| 2 | 1990 - 2000 | 325 |
| 3 | 1970 - 1990 | 8 |
| 4 | 1970年以前 | 1 |

6.5 分析二手房的层数情况并导出到 mysql 中的 house 数据库

```
SELECT
 a.cengshu,
 count(*)
FROM
 (
  SELECT
      CASE
  WHEN cengshu IN ('中层', '低层', '高层') THEN
      cengshu
  ELSE
      '其他'
  END AS cengshu
  FROM
      house_table
 ) AS a
GROUP BY
 a.cengshu
""").toPandas()

pd.io.sql.to_sql(
    df_cengshu,
    name='cengshu',
    con=conn,
    schema='house',
    if_exists='replace',
    index=True,
    index_label='id'
)

```

运行结果：

开始事务 备注 筛选 排序 导入 导出

| id | cengshu | num |
|---|---|---|
| 0 | 低层 | 1173 |
| 1 | 中层 | 899 |
| 2 | 高层 | 375 |
| 3 | 其他 | 193 |

6.6 分析分词情况并导出到 mysql 中的 house 数据库

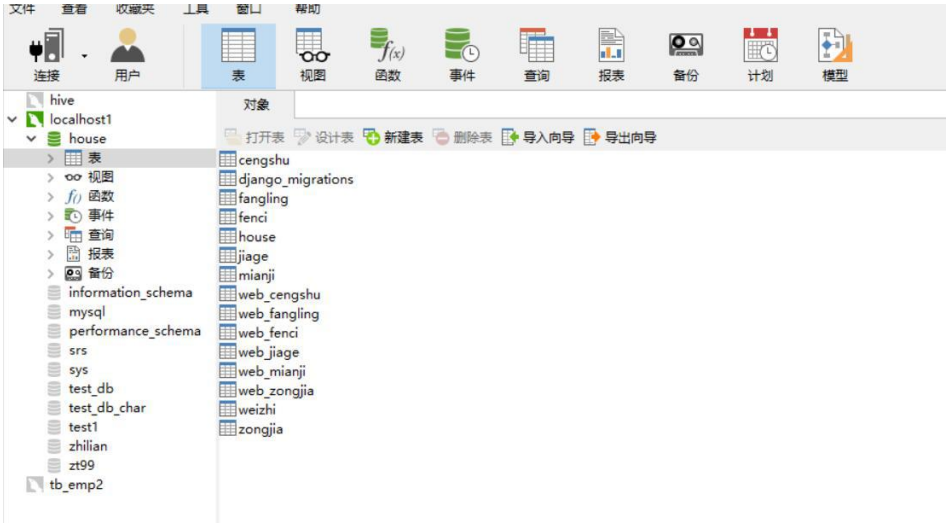
```
INSERT web_fenci(fenci,num) SELECT fenci,num FROM fenci
pd.io.sql.to_sql(
    df_fenci,
    name='fenci',
    con=conn,
    schema='house',
    if_exists='replace',
    index=True,
    index_label='id'
)
```

运行结果：

127.0.0.1:8000/fenci_data/

[{"name": "精装", "value": 505}, {"name": "通透", "value": 397}, {"name": "三室", "value": 345}, {"name": "随时", "value": 315}, {"name": "急售", "value": 278}, {"name": "老证", "value": 246}, {"name": "钥匙", "value": 244}, {"name": "捡漏", "value": 234}, {"name": "出售", "value": 227}, {"name": "精装修", "value": 218}, {"name": "小区", "value": 215}, {"name": "户型", "value": 213}, {"name": "楼层", "value": 209}, {"name": "采光", "value": 186}, {"name": "两室", "value": 184}, {"name": "拎包", "value": 182}, {"name": "诚心", "value": 182}, {"name": "装修", "value": 176}, {"name": "房东", "value": 176}, {"name": "毛坯", "value": 173}, {"name": "花园", "value": 169}, {"name": "一楼", "value": 169}, {"name": "入住", "value": 166}, {"name": "建业", "value": 164}, {"name": "平米", "value": 163}, {"name": "地下室", "value": 152}, {"name": "万元", "value": 150}, {"name": "精装", "value": 505}, {"name": "看房", "value": 447}, {"name": "南北", "value": 422}, {"name": "通透", "value": 397}, {"name": "单价", "value": 386}, {"name": "电梯", "value": 355}, {"name": "三室", "value": 345}, {"name": "随时", "value": 315}, {"name": "急售", "value": 278}, {"name": "老证", "value": 246}, {"name": "钥匙", "value": 244}, {"name": "捡漏", "value": 234}, {"name": "出售", "value": 227}, {"name": "精装修", "value": 218}, {"name": "小区", "value": 215}, {"name": "户型", "value": 213}, {"name": "楼层", "value": 209}, {"name": "采光", "value": 186}, {"name": "两室", "value": 184}, {"name": "拎包", "value": 182}, {"name": "诚心", "value": 182}, {"name": "装修", "value": 176}, {"name": "房东", "value": 176}, {"name": "毛坯", "value": 173}, {"name": "花园", "value": 169}, {"name": "一楼", "value": 169}, {"name": "入住", "value": 166}, {"name": "建业", "value": 164}, {"name": "平米", "value": 163}, {"name": "地下室", "value": 152}, {"name": "万元", "value": 150}, {"name": "南北", "value": 909}, {"name": "通透", "value": 656}, {"name": "满五", "value": 560}, {"name": "精装修", "value": 509}, {"name": "看房", "value": 496}, {"name": "采光", "value": 377}, {"name": "地铁", "value": 347}, {"name": "小区", "value": 343}, {"name": "出售", "value": 330}, {"name": "楼层", "value": 328}, {"name": "电梯", "value": 314}, {"name": "房东", "value": 305}, {"name": "钥匙", "value": 284}, {"name": "随时", "value": 276}, {"name": "户型", "value": 275}, {"name": "急售", "value": 269}, {"name": "两房", "value": 261}, {"name": "三房", "value": 247}, {"name": "住房", "value": 243}, {"name": "精装", "value": 231}, {"name": "诚意", "value": 224}, {"name": "位置", "value": 219}, {"name": "地铁口", "value": 213}, {"name": "诚心", "value": 211}, {"name": "业主", "value": 200}, {"name": "方便", "value": 198}, {"name": "装修", "value": 180}, {"name": "置换", "value": 179}, {"name": "价格", "value": 179}, {"name": "车位", "value": 166}, {"name": "朝南", "value": 157}]

## 运行结果

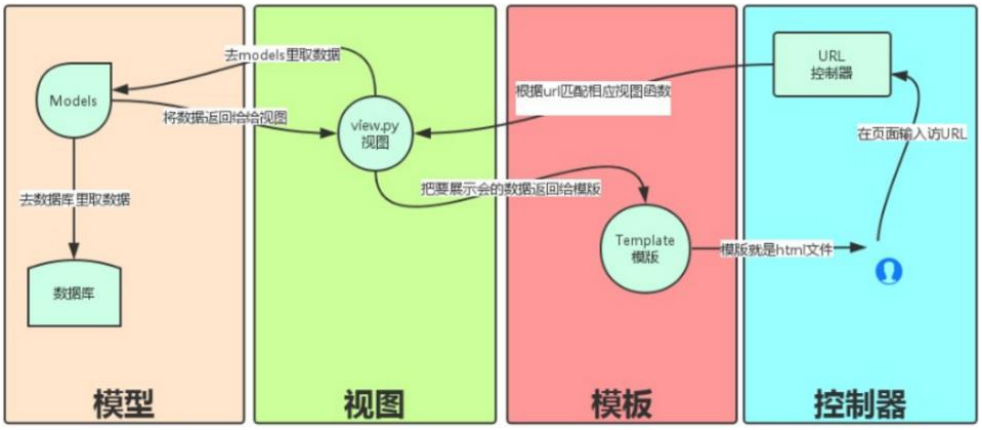



# 2.5 数据可视化

## 原理

MVC 模式将应用程序分解成三个组成部分:model(模型),view(视图),和 controller(控制器)。Django 也是一个 MVC 框架。但是在 Django 中，控制器接受用户输入的部分由框架自行处理，所以 Django 里更关注的是模型（Model）、模板(Template)和视图（Views），称为 MTV 模式：M 代表模型（Model），即数据存取层。T 代表模板(Template)，即表现层。

V 代表视图（View），即业务逻辑层。 Django 原理流程图如下：




目录结构

- bar：外层的 bar 根目录仅仅是项目的一个容器。它的命名对 Django 无关紧要；你可以把它重新命名为任何你喜欢的名字。
- templates：html 页面存放路径，在 Django 中叫模板，通过固定的语法把模板渲染后，才是真正的 html 页面。
- 内层的 bar/目录是你的项目的真正的 Python 包。它是你导入任何东西时将需要使用的 Python 包的名字（例如 bar.urls）。
- bar/settings.py：该 Django 项目的设置/配置。Django 设置 将告诉你这些设置如何工作。
- bar/urls.py：该 Django 项目的 URL 声明；你的 Django 站点的“目录”。
- bar/wsgi.py：用于你的项目的与 WSGI 兼容的 Web 服务器入口。
- 内层的 web/目录是创建项目时添加的一个 app。
- web/admin.py 是 Django 强大的管理系统，详细可以查看 Django 官方文档。
- web/models.py 是 Django 为 PostgreSQL、MySQL、SQLite、Oracle 提供了统一的调用 API。
- web/views.py 是用来处理程序逻辑，然后呈现到 template(一般为 GET 方法, POST 方法略有不同)。
- manage.py：一个命令行工具，可以使你用多种方式对 Django 项目进行交互。 你可以在 django-admin 和 manage.py 中读到关于 manage.py 的所有细节。

Django Model 介绍

每一个 Django Model 都继承自 django.db.models.Model。

在 Model 当中每一个属性 attribute 都代表一个 database field。

通过 Django Model API 可以执行数据库的增删改查，而不需要写一些数据库的查询语句。

网页程序的逻辑

request 进来=>从服务器获取数据->处理数据->把网页呈现出来。

url 设置相当于客户端向服务器发出 request 请求的入口，并用来指明要调用的程序逻辑

views 用来处理程序逻辑，然后呈现到 template(一般为 GET 方法, POST 方法略有不同)

template 一般为 html+CSS 的形式，主要是呈现给用户的表现形式

# 操作步骤

**准备工作：**

修改 setting 文件

1）将 DATABASES 里面的内容设置为修改我们自己 MySQL 的数据库设置，包括数据库名称、用户名以及密码。

2）将'django.middleware.csrf.CsrfViewMiddleware'这一行注释掉， CSRF 是“跨站请求伪造”；所以要先将先将其注释，这样可以顺利通过 post 请求，

3）将 static 绝对路径赋值给 STATICFILES_DIRS，这样 static 目录下存放的 js 和样式文件，可以 static 的相对路径导入，保证我们的 css、js 与 html 可以正常导入。

## 1. 使用词云可视化

有了前一步的 Jieba 分词的步骤，我们能够得到分词后的数据并将其存放在数据库中。在 house_jieba 中仅有两列数据，即 skill_name 和 num，因此在 Models 中定义对应的 name 和 values 用于返回数据，

```
class Fenci(models.Model):
    Fenci = models.TextField(db_column='Fenci', blank=True, null=True)
    num = models.IntegerField(db_column='num', blank=True, null=True)
```

在 view 向 Models 发送请求时将数据返回。另外添加词云对应的 url，即

```
path('fenci_data/', views.fenci_data),
    path('fenci/', views.fenci),
```

根据匹配的 url 访问视图函数，view 利用数据库查询语句将 house_jieba 的数据返回给模板

```
ef fenci_data(request):
    data_dict = []
    items = Fenci.objects.filter(num__gte=150)
    for item in items:
        data_dict.append({
            'name':item.Fenci,
            'value':item.num,
        })
    return HttpResponse(json.dumps(data_dict,ensure_ascii=False))
def fenci(request):
    return render(request,'word_cloud.html')
```

最后，通过 word_cloud.html 将词云展示到网页

```html
<!DOCTYPE html>
<!-- saved from url=(0037)http://v3.bootcss.com/examples/cover/ -->
<html lang="zh-CN">
<head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">

    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <!-- 上述 3 个 meta 标签*必须*放在最前面，任何其他内容都*必须*跟随其后！ -->
    <meta name="description" content="">
    <meta name="author" content="">

    <title>Visualization</title>

    <!-- Bootstrap core CSS -->
    <link href="../static/bootstrap.min.css" rel="stylesheet">

    <!-- Custom styles for this template -->
    <link href="../static/cover.css" rel="stylesheet">

</head>

<body>

<div class="site-wrapper">


    <div class="row">

        <div class="center-block">
            <div>
                <h3 class="masthead-brand"></h3>
            </div>
        </div>

        <div>
            {#               <h1 class="cover-heading">2017 年上半年热销 SUV</h1>#}

            <div class="center-block" id="main" style="width: 700px;height:500px;"></div>
            <p class="lead">2017 年上海二手房关注词</p>
            <p class="lead">利用字符云的形式展示。</p>
        </div>
```

```
43.
44.         <div class="mastfoot">
45.
46.         </div>
47.
48.     </div>
49.
50.
51. </div>
52.
53. <!-- Bootstrap core JavaScript
54. ================================================== -->
55. <!-- Placed at the end of the document so the pages load faster -->
56. <script src="../static/jquery.min.js"></script>
57. <script src="../static/bootstrap.min.js"></script>
58. <script src="/static/echarts-all.js"></script>
59. <script src="/static/dark.js"></script>
60.
61.
62. <script type="text/javascript">
63.     $.get('/fenci_data/', function (data) {
64.         var myChart = echarts.init(document.getElementById('main'), 'dark');
65.
66.         function createRandomItemStyle() {
67.             return {
68.                 normal: {
69.                     color: 'rgb(' + [
70.                         Math.round(Math.random() * 390),
71.                         Math.round(Math.random() * 390),
72.                         Math.round(Math.random() * 390)
73.                     ].join(',') + ')'
74.                 }
75.             };
76.         }
77.
78.         for (var key in data) {
79.             data[key]['itemStyle'] = createRandomItemStyle();
80.         }
81.
82.         option = {
83.             title: {
84.                 text: '2017年上海二手房关注词',
85.                 // link: 'http://www.google.com/trends/hottrends'
86.
```

```
            },
            tooltip: {
                show: true
            },
            series: [{
                name: '数量',
                type: 'wordCloud',
                size: ['80%', '80%'],
                textRotation: [0, 45, 90, -45],
                textPadding: 0,
                autoSize: {
                    enable: true,
                    minSize: 14
                },
                data: data
            }]
        };
        myChart.setOption(option);
    }, "json");


</script>
</body>
</html>
```

运行结果：

## 2. 二手房面积分布可视化

在数据库中，二手房价格为 mianji，通过分析，两种分布情况能够通过饼状图展示的更

加直观具体，并将两图展示在同一页面上，两个表对应的仅有两列数据，因此 Models 中的 name 和 values 不需要改变，只需加上两个表分别对应的 urls 以及总表的 urls,即

```
path('mianji_data/', views.mianji_data),
    path('mianji/', views.mianji),
```

以及对应的 views,但是将两个表同时以饼状图展示到一个网页中，就要对每个表先建立一个 view 单个查询,再建立一个总的 view 访问所有数据,如下步骤

```
 def mianji_data(request):
    data_dict = []
    items = Mianji.objects.all()
    for item in items:
        data_dict.append({
            'name':item.mianji,
            'value':item.num,

        })
    return HttpResponse(json.dumps(data_dict))

def mianji(request):
    return render(request,'mianji.html')
```

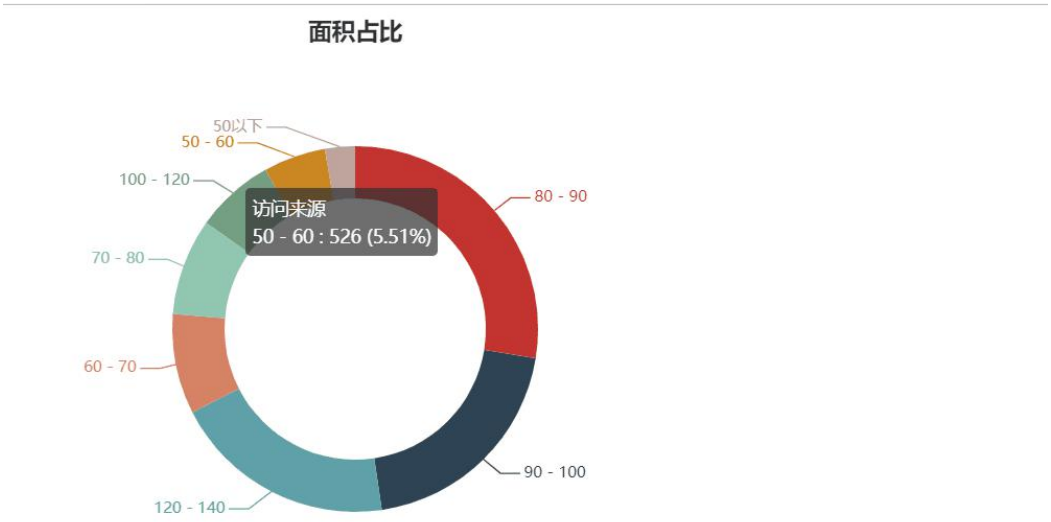
最后，同样创建 mianji.html，以饼环状图展示在网页上

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>ECharts</title>
</head>
<body>
<!-- 为 ECharts 准备一个具备大小（宽高）的 DOM -->
<div id="main" style="width: 600px;height:400px; float: left"></div>
{#<script src="../static/jquery.min.js"></script>#}
{#<script src="../static/bootstrap.min.js"></script>#}
{#<script src="../static/echarts-all.js"></script>#}
<script src="../static/js/echarts.min.js"></script>
<script src="../static/js/jquery-3.5.0.js"></script>
{#<script src="../static/dark.js"></script>#}
<script>
    var myChart = echarts.init(document.getElementById('main'));
    var option = {
        title : {
```

```
            text: '面积占比',
            subtext: ' ',
            x:'center'
        },
        tooltip : {
            trigger: 'item',
            formatter: "{a} <br/>{b} : {c} ({d}%)"
        },
        legend: {
            orient: 'vertical',
            left: 'left',
        },
        series : [
            {
                name: '访问来源',
                type: 'pie',
                radius : ['50%', '70%'],
                center: ['50%', '60%'],
                data:(function () {
                    var datas = [];
                    $.ajax({
                        type:"GET",
                        url:"/mianji_data/",
                        dataType:"json",
                        async:false,
                        success:function (result) {
                            console.log(result)
                            for (var i = 0; i < result.length; i++){
                                datas.push({
                                    "value":result[i].value,
"name":result[i].name
                                })
                            }
                            console.log(datas)
                        }
                    });
                    return datas;
                })(),
                itemStyle: {
                    emphasis: {
                        shadowBlur: 10,
                        shadowOffsetX: 0,
                        shadowColor: 'rgba(0, 0, 0, 0.5)'
                    }
```

```
                }
            }
        ]
    };
    myChart.setOption(option);
</script>
</body>
</html>
```

**运行结果：**




### 3. 二手房价格分布情况可视化

在数据库中，二手房价格为 jiage。通过分析，我们选用环形图展示经验分布情况，添加对应的 urls，即

```
path('jiage_data/', views.jiage_data),
    path('jiage/', views.jiage),
```

以及对应的 views，即

```
def jiage_data(request):
    data_dict = []
    items = Jiage.objects.all()
    for item in items:
        data_dict.append({
```

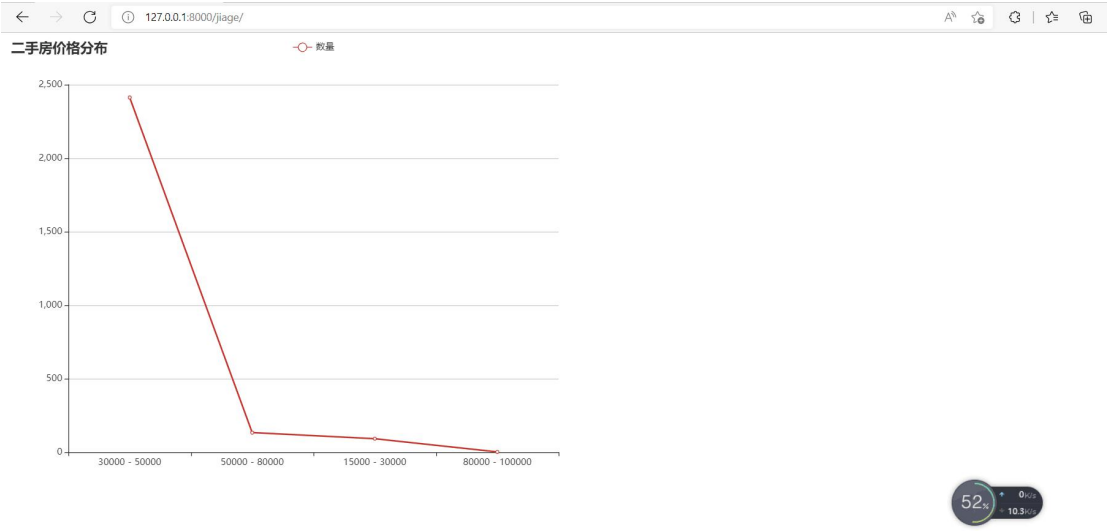
```
            'name':item.Jiage,
            'value':item.num,
        })
    return HttpResponse(json.dumps(data_dict))
def jiage(request):
    return render(request,'jiage.html')
```

创建 jiage.html，以可缩放柱状图形式展示在网页

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>ECharts</title>
    <script src="../static/js/echarts.min.js"></script>
    <script src="../static/js/jquery-3.5.0.js"></script>
</head>
<body>
<!-- 为 ECharts 准备一个具备大小（宽高）的 DOM -->
<div id="main" style="width: 800px;height:600px;"></div>

<script type="text/javascript">
    // 基于准备好的dom，初始化echarts实例
    var myChart = echarts.init(document.getElementById('main'));

    // 新建productName与nums数组来接受数据，因为我们
    var jiageName = [];
    var nums = [];

    //AJAX接收数据主体
    $.ajax({
        type:"GET",
        url:"/jiage_data/",
        dataType:"json",
        async:false,
        success:function (result) {

            for (var i = 0; i < result.length; i++){
                jiageName.push(result[i].name);
                nums.push(result[i].value);
            }

        },
        error :function(errorMsg) {
```

```
            alert("获取后台数据失败！");
        }
    });

    // 指定图表的配置项和数据
    var option = {
        title: {
            text: '二手房价格分布'
        },
        tooltip: {},
        legend: {
            data:['数量']
        },
        xAxis: {
            //结合
            data: jiageName
        },

        yAxis: {},
        series: [{
            name: '数量',
            type: 'line',
            //结合
            data: nums
        }]
    };

    // 使用刚指定的配置项和数据显示图表。
    myChart.setOption(option);
</script>
</body>
</html>
```

**运行结果：**




## 4. 二手房房量情况可视化

在数据库中，二手房房量为 fangliang，通过分析，我们选用折线图将低中高层及其他情况展示出来

对于多列数据的情况，就需要在 Models 中添加对应数据的属性，即

```
class Fangling(models.Model):
    id = models.BigIntegerField(primary_key=True)
    fangling = models.TextField(db_column='fangling', blank=True, null=True)
    num = models.BigIntegerField(db_column='num', blank=True, null=True)
```

添加对应的 urls，即

```
path('fangling_data/', views.fangling_data),
    path('fangling/', views.fangling),
```

添加对应的 views，之后添加对应于 Models 的每个属性值，代码如下：

```
def fangling_data(request):
    data_dict = []
    items = Fangling.objects.all()
    for item in items:
        data_dict.append({
            'name':item.fangling,
            'value':item.num,
        })
    return HttpResponse(json.dumps(data_dict))

```

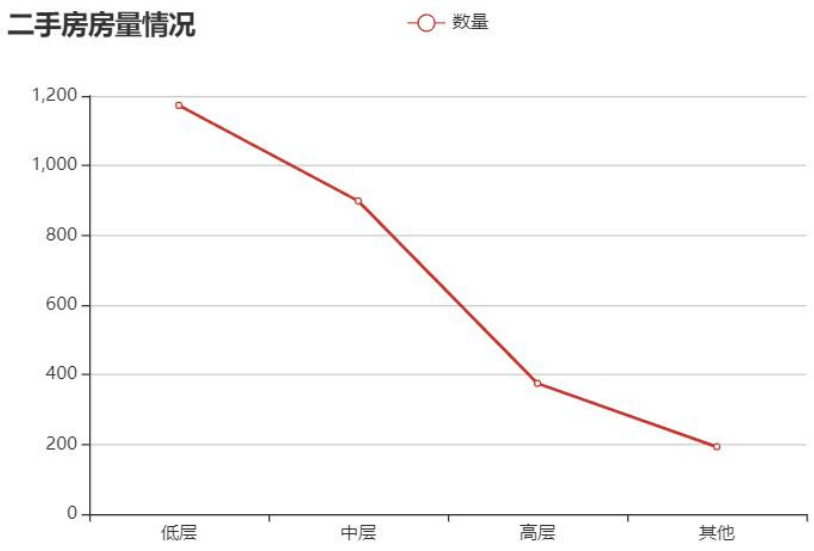
```
def fangling(request):
    return render(request, 'fangling.html')
```

最后创建 fangliang.html，以数组的形式将获得的每列数据输入对应的图表中，代码如下：

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>ECharts</title>
    <script src="../static/js/echarts.min.js"></script>
    <script src="../static/js/jquery-3.5.0.js"></script>
</head>
<body>
<!-- 为 ECharts 准备一个具备大小（宽高）的 DOM -->
<div id="main" style="width: 800px;height:600px;"></div>

<script type="text/javascript">
    // 基于准备好的dom，初始化echarts实例
    var myChart = echarts.init(document.getElementById('main'));

    // 新建productName与nums数组来接受数据，因为我们
    var fanglingName = [];
    var nums = [];

    //AJAX接收数据主体
    $.ajax({
        type:"GET",
        url:"/fangling_data/",
        dataType:"json",
        async:false,
        success:function (result) {

            for (var i = 0; i < result.length; i++){
                fanglingName.push(result[i].name);
                nums.push(result[i].value);
            }

        },
        error :function(errorMsg) {
            alert("获取后台数据失败！");
        }
    });

```

```
    // 指定图表的配置项和数据
    var option = {
        title: {
            text: '二手房房量情况'
        },
        tooltip: {},
        legend: {
            data:['数量']
        },
        xAxis: {
            //结合
            data: fanglingName
        },

        yAxis: {},
        series: [{
            name: '数量',
            type: 'bar',
            //结合
            data: nums
        }]
    };

    // 使用刚指定的配置项和数据显示图表。
    myChart.setOption(option);
</script>
</body>
</html>
```

**运行结果：**




5. 二手房总价分布可视化

在数据库中，二手房总价为 zongjia，我们选用环形图。Models 中仍然采用 name 和 values，只需添加对应的 urls，即

```
path('zongjia_data/', views.zongjia_data),
    path('zongjia/', views.zongjia),
```

添加对应的 views，

```
def zongjia_data(request):
    data_dict = []
    items = Zongjia.objects.all()
    for item in items:
        data_dict.append({
            'name':item.zongjia,
            'value':item.num,
        })
    return HttpResponse(json.dumps(data_dict))

def zongjia(request):
    return render(request,'zongjia.html')
```

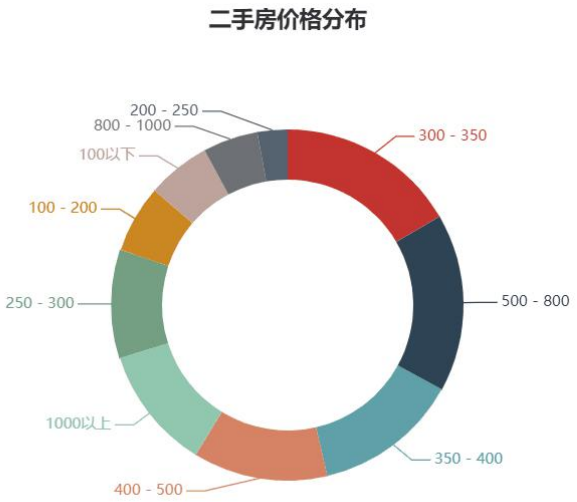
最后创建 zongjia.html 将二手房价格的环形图展示到网页

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>ECharts</title>
</head>
<body>
<!-- 为 ECharts 准备一个具备大小（宽高）的 DOM -->
<div id="main" style="width: 600px;height:400px; float: left"></div>
{#<script src="../static/jquery.min.js"></script>#}
{#<script src="../static/bootstrap.min.js"></script>#}
{#<script src="../static/echarts-all.js"></script>#}
<script src="../static/js/echarts.min.js"></script>
<script src="../static/js/jquery-3.5.0.js"></script>
{#<script src="../static/dark.js"></script>#}
<script>
    var myChart = echarts.init(document.getElementById('main'));
    var option = {
        title : {
            text: '二手房价格分布',
            subtext: ' ',
            x:'center'
        },
        tooltip : {
            trigger: 'item',
            formatter: "{a} <br/>{b} : {c} ({d}%)"
        },
        legend: {
            orient: 'vertical',
            left: 'left',
        },
        series : [
            {
                name: '访问来源',
                type: 'pie',
                radius : ['50%', '70%'],
                center: ['50%', '60%'],
                data:(function () {
                    var datas = [];
                    $.ajax({
                        type:"GET",
                        url:"/zongjia_data/",
                        dataType:"json",
                        async:false,
                        success:function (result) {
                            console.log(result)
                            for (var i = 0; i < result.length; i++){
```

```
                                datas.push({
                                    "value":result[i].value,
    "name":result[i].name
                                })
                            }
                            console.log(datas)
                        }
                    });
                    return datas;
                })(),
                itemStyle: {
                    emphasis: {
                        shadowBlur: 10,
                        shadowOffsetX: 0,
                        shadowColor: 'rgba(0, 0, 0, 0.5)'
                    }
                }
            }
        ]
    };
    myChart.setOption(option);
</script>
</body>
</html>
```

**运行结果：**




## 6. 二手房房龄分布可视化

在数据库中，对应使用的是 fangling，通过分析，我们选择将房龄的数据绘制成柱状图，

Models 的 name 和 values 仍然使用，要添加分别对应的两个表的 urls 以及总表的 urls,即

```
path('fangling_data/', views.fangling_data),
    path('fangling/', views.fangling),
```

添加两个表的 views 和总表的 views，

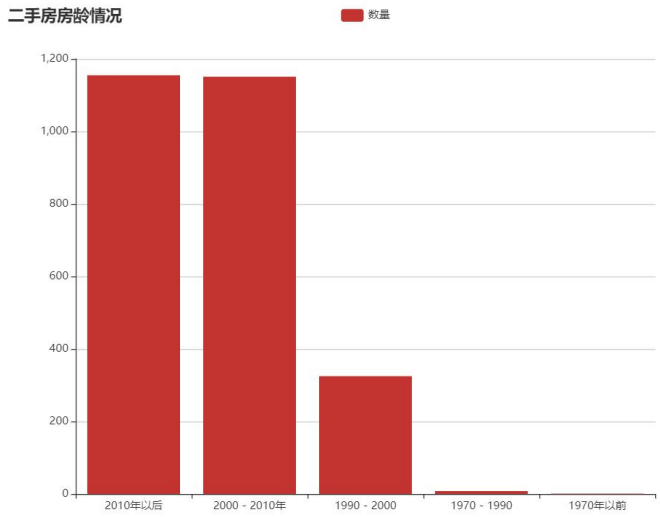
```
def fangling_data(request):
    data_dict = []
    items = Fangling.objects.all()
    for item in items:
        data_dict.append({
            'name':item.fangling,
            'value':item.num,
        })
    return HttpResponse(json.dumps(data_dict))

def fangling(request):
    return render(request, 'fangling.html')
```

最后创建 fangling.html，将柱状图放在该页面展示

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>ECharts</title>
    <script src="../static/js/echarts.min.js"></script>
    <script src="../static/js/jquery-3.5.0.js"></script>
</head>
<body>
<!-- 为 ECharts 准备一个具备大小（宽高）的 DOM -->
<div id="main" style="width: 800px;height:600px;"></div>

<script type="text/javascript">
    // 基于准备好的 dom，初始化 echarts 实例
    var myChart = echarts.init(document.getElementById('main'));

    // 新建 productName 与 nums 数组来接受数据，因为我们
    var fanglingName = [];
    var nums = [];

    //AJAX 接收数据主体
    $.ajax({
        type:"GET",
```

```
        url:"/fangling_data/",
        dataType:"json",
        async:false,
        success:function (result) {

            for (var i = 0; i < result.length; i++){
                fanglingName.push(result[i].name);
                nums.push(result[i].value);
            }

        },
        error :function(errorMsg) {
            alert("获取后台数据失败！");
        }
    });

    // 指定图表的配置项和数据
    var option = {
        title: {
            text: '二手房房量情况'
        },
        tooltip: {},
        legend: {
            data:['数量']
        },
        xAxis: {
            //结合
            data: fanglingName
        },

        yAxis: {},
        series: [{
            name: '数量',
            type: 'bar',
            //结合
            data: nums
        }]
    };

    // 使用刚指定的配置项和数据显示图表。
    myChart.setOption(option);
</script>
</body>
</html>
```

**运行结果：**




## 2.6 数据挖掘分析

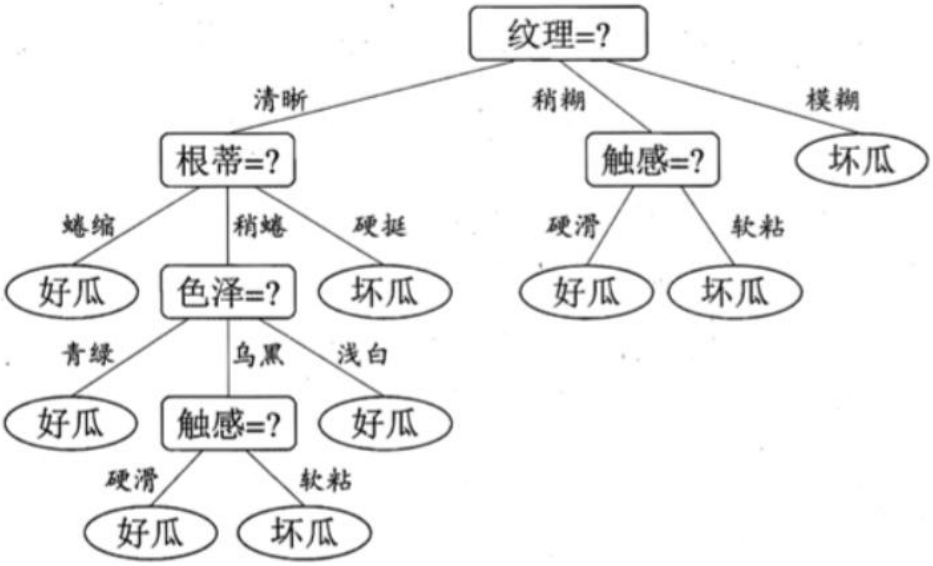
## 原理

本实验通过 PySpark MLlib 决策树回归算法与朴素贝叶斯算法对 58 同城有关数据分析岗位的招聘信息探索，项目背景为：对于数据分析工作岗位来说，不同的公司规模、公司融资阶段、公司所在的省份、以及工作岗位所要求的工作经验与学历要求，都会对岗位的薪资产生影响，本实验首先通过朴素贝叶斯算法对招聘信息及其薪资进行分类，再通过决策树算法对不同情况下的薪资进行预测。

朴素贝叶斯分类是一种十分简单的分类算法，朴素贝叶斯的思想基础为：对于给出的待分类项，求解在此项出现的条件下各个类别出现的概率，哪个最大，就认为此待分类项属于哪个类别。通俗来说，就好比，你在街上看到一个黑人，十有八九会觉得这个黑人来自非洲。因为黑人中非洲人的比率最高，当然也可能是美洲人或亚洲人，但在没有其它可用信息下，我们会选择条件概率最大的类别，这就是朴素贝叶斯的思想基础。用公式表示如下：

$$\mathrm{p}(\text{类别}|\text{特征})=\frac{p(\text{特征}|\text{类别})\mathrm{p}(\text{类别})}{\mathrm{p}(\text{特征})}$$

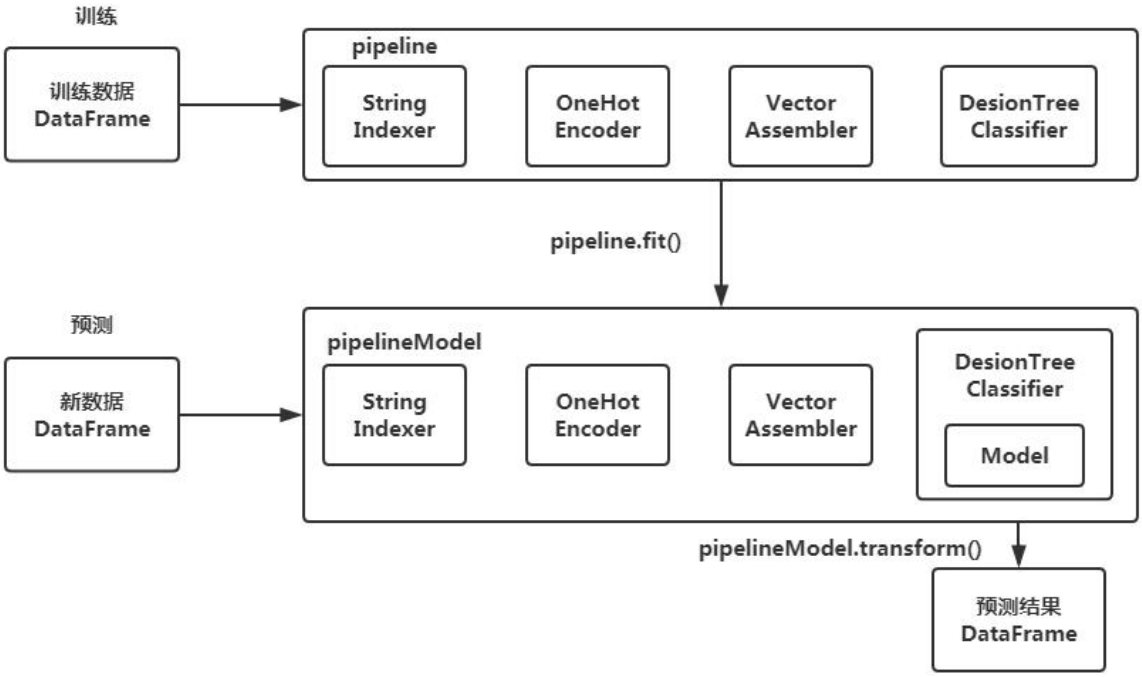
决策树是基于树结构进行决策的，其机制就是通过判定每个属性分类的纯度来进行自上而下决策分类，决策树包含根结点，内部结点，叶结点；根结点和内部结点对应与分类的属性(也就是分类的基准)，叶结点对应决策结果(也就是纯度很高且不需要继续分裂的类别)；从根结点到某一个叶结点的路径便是当前叶结点对应类的整个决策过程。下面是一个基于西瓜数据集来判断是好瓜还是坏瓜而生成的决策树。




# 操作步骤

## 原理

Spark 机器学习工作流程（ML Pipeline）的原理就是将机器学习的每一个阶段（例如数据处理、进行训练与测试、建立 Pipeline 流程）形成机器学习工作流程，流程图如下：




（1）建立机器学习流程 pipeline：包含 4 个阶段，前 3 个是数据处理，第 4 阶段是 DesionTreeClassifier 机器学习分类算法

StringIndexer：将文字的分类特征字段转化为数字

OneHotEncoder：将一个数字的分类特征字段转为多个字段

VectorAssembler：将所有的特征字段整合成一个 Vector 字段

DesionTreeClassifier：进行训练并且产生模型

（2）训练：“训练数据 DataFrame”使用 pipeline.fit()进行训练。系统会按照顺序执行每一个阶段，最后产生 pipelineModel 模型。pipelineModel 与 pipeline 类似，只是多了训练后建立的模型 Model

（3）预测："新数据 DataFrame"使用 pipelineModel.trainsform()进行预测。系统会按照顺序执行每一个阶段，并使用 DecisionTree Classifier Model 进行预测。预测完成后，会产生“预测结果 DataFrame”

Spark 机器学习流程（ML Pipeline）的名词说明如下：

DataFrame：Spark ML 机器学习 API 处理的数据格式是 DataFrame，我们必须使用 DataFrame 存储训练数据、测试数据，最后预测结果也是 DataFrame，我们可以使用 sqlContext 读取文本文件创建 DataFrame，或将 RDD 转换为 DataFrame，也可以使用 Spark SQL 来操作，DataFrame 可以存储不同的数据类型，文字，特征字段所创建的向量 vectors、label 标签字段。

Transformer：是一个算法，可以使用 transform 方法将一个 DataFrame 转换为另一个 DataFrame。

Estimator：是一个算法，可以使用 fit 方法传入 DataFrame，产生一个 Transformer。

Pipeline：可以串联多个 Transformers 与 Estimator 建立 ML 机器学习的 Workflow 工作流程。

Parameter：以上 Transformers 与 Estimator 都可以共享相同的 Parameter API。

利用.cast("double")转换为 double

.alias(column)把别名设置为原来的字段名

```
df= row_df.select(
                    ['floor','location','age']+
                    [col(column).cast("double").alias(column)
                     for column in row_df.columns[0:5] ] )
```

```
root
 |-- floor: string (nullable = true)
 |-- location: string (nullable = true)
 |-- age: string (nullable = true)
 |-- price: double (nullable = true)
 |-- room: double (nullable = true)
 |-- living: double (nullable = true)
 |-- rest: double (nullable = true)
 |-- area: double (nullable = true)
```

在建立 pipeline 数据处理流程之前，我们先介绍将用来建立流程的组件

StringIndexer：将文字的分类特征字段转化为数字

OneHotEncoder：将一个数字的分类特征字段转为多个字段的 Vector

VectorAssembler：将所有的特征字段整合成一个 Vector 的特征字段

DesionTreeClassifier：决策树分类

**StringIndexer**

StringIndexer 的功能类似于 categoriesMap，是网页分类特征字段的字典，可用于将字符串分类特征字段转换为数值，StringIndexer 是一个“Estimator”，所以使用上必须分为两个步骤

（1）使用 fit 方法传入参数 DataFrame，产生一个“Transformer”

（2）针对生成的“Transformer”，使用 transform 方法将 DataFrame 转换为另一个 DataFrame

```
print (df.printSchema())
print(df.select('room','living','rest','area','floor','age','location','price').show(10))
train_df, test_df = df.randomSplit([0.7, 0.3])
train_df.cache()
test_df.cache()
floorIndexer = StringIndexer(
                                        inputCol='floor',
                                        outputCol="floor_Index")
floorTransformer=floorIndexer.fit(df)
for i in range(0,len(floorTransformer.labels)):
    print (str(i)+':'+floorTransformer.labels[i])
df1=floorTransformer.transform(df)
print (df1.columns)
df1.select("floor","floor_Index").show(10)
from pyspark.ml.feature import   OneHotEncoder
encoder = OneHotEncoder(dropLast=False,
                                inputCol='floor_Index',
                                outputCol="floor_IndexVec")
df2=encoder.transform(df1)
print (df2.columns)
df2.select("floor","floor_Index",
                        "floor_IndexVec").show(10)


# assemblerInputs =['price', 'room', 'living', 'rest', 'area','floor_IndexVec']
# print (assemblerInputs)
# assembler = VectorAssembler(
```

```
#                                       inputCols=assemblerInputs,
#                                       outputCol="features")
# df22=assembler.transform(df2)
# print (df22.columns)
# df22.select('features').show(1)

locationIndexer = StringIndexer(
                                            inputCol='location',
                                            outputCol="location_Index")
locationTransformer=locationIndexer.fit(df2)
for i in range(0,len(locationTransformer.labels)):
      print (str(i)+':'+locationTransformer.labels[i])
df3=locationTransformer.transform(df2)
print (df3.columns)
df3.select("location","location_Index").show(10)
from pyspark.ml.feature import   OneHotEncoder
encoder = OneHotEncoder(dropLast=False,
                                      inputCol='location_Index',
                                      outputCol="location_IndexVec")
df4=encoder.transform(df3)
print (df4.columns)
df4.select("location","location_Index",
                            "location_IndexVec").show(10)
#
# assemblerInputs =['location_IndexVec','features']
# print (assemblerInputs)
# assembler = VectorAssembler(
#                                       inputCols=assemblerInputs,
#                                       outputCol="features1")
# df44=assembler.transform(df4)
# print (df44.columns)
#
#
#
ageIndexer = StringIndexer(
```

```
                                        inputCol='age',
                                        outputCol="age_Index")
ageTransformer=ageIndexer.fit(df4)
for i in range(0,len(ageTransformer.labels)):
    print (str(i)+':'+ageTransformer.labels[i])
df5=ageTransformer.transform(df4)
print (df5.columns)
df5.select("age","age_Index").show(10)
from pyspark.ml.feature import   OneHotEncoder
encoder = OneHotEncoder(dropLast=False,
                                inputCol='age_Index',
                                outputCol="age_IndexVec")
df6=encoder.transform(df5)
print (df6.columns)
df6.select("age","age_Index",
                        "age_IndexVec").show(10)
assemblerInputs =[ 'price','room', 'living', 'rest',
'area','floor_IndexVec','location_IndexVec','age_IndexVec']
print (assemblerInputs)

assembler = VectorAssembler(
                                inputCols=assemblerInputs,
                                outputCol="features")
df66=assembler.transform(df6)
print (df66.columns)
df66.select('features').show(10)
```

```
['price', 'room', 'living', 'rest', 'area', 'floor_IndexVec', 'location_IndexVec', 'age_IndexVec']
['floor', 'location', 'age', 'price', 'room', 'living', 'rest', 'area', 'floor_Index', 'floor_IndexVec', 'location_Index', 'location_IndexVec', 'age_Index', 'age_IndexVec', 'features']
+--------------------+
|            features|
+--------------------+
|(59,[0,1,2,3,4,6,...|
|(59,[0,1,2,3,4,5,...|
|(59,[0,1,2,3,4,7,...|
|(59,[0,1,2,3,4,7,...|
|(59,[0,1,2,3,4,6,...|
|(59,[0,1,2,3,4,6,...|
|(59,[0,1,2,3,4,5,...|
|(59,[0,1,2,3,4,6,...|
|(59,[0,1,2,3,4,5,...|
|(59,[0,1,2,3,4,6,...|
+--------------------+
```

接下来查看数据规模和查看数据集的统计数据,包括平均值，标准差，数量统计等

```
# 3-探索分析数据
print('--------------  探索分析数据  -----------------')
print((df66.count(), len(df66.columns)))                        #  查看数据规模
```

df66.printSchema()  # 查看数据结构类型

df66.describe().show(5,False)  # 查看数据集的统计数据,包括平均值，标准差，数量统计等。

from pyspark.sql.functions import corr

# df.select(corr('var_1','output')).show()  # 计算数据方差

```
+-------+-----+--------+-----------------+------------------+-------------------+-------------------+-------------------+-------------------+-------------------+-------------------+-------------------+
|summary|floor|location|age              |price             |room               |living             |rest               |area               |floor_Index        |location_Index     |age_Index          |
+-------+-----+--------+-----------------+------------------+-------------------+-------------------+-------------------+-------------------+-------------------+-------------------+-------------------+
|count  |2640 |2640    |2640             |2640              |2640               |2640               |2640               |2640               |2640               |2640               |2640               |
|mean   |null |null    |2007.782196969697|353.7470227727285|2.3367424242424244|1.8223484848484848 |1.058712121212121  |88.49261363636363|0.843939393939394 |1.6681818181818182|8.529924242424242|
|stddev |null |null    |6.432903861165169|48.59726201163909 6|0.4876791884510068|0.4016267788553507 5|0.2461523704340460 3|5.637809013783927|0.9245305498010101|1.8664513112946732|7.0443739422528 11|
|min    |中层 |滨定    |1966             |270.0             |1.0                |0.0                |0.0                |42.7               |0.0                |0.0                |0.0                |
|max    |高层 |黄浦    |2021             |440.0             |4.0                |3.0                |2.0                |100.0              |3.0                |13.0               |35.0               |
+-------+-----+--------+-----------------+------------------+-------------------+-------------------+-------------------+-------------------+-------------------+-------------------+-------------------+


(1841, 15)
(799, 15)
```

然后进行建模预测

from pyspark.ml.linalg import Vector

from pyspark.ml.feature import VectorAssembler  # 导入库 VectorAssembler

# print('----------  数据转换  ----------')

# vec_assmebler=VectorAssembler(inputCols=['price'],outputCol='features1') # 转换，这里相对将多元一次方程中的各变量存放到一个向量中

# features_df=vec_assmebler.transform(df66)

# features_df.printSchema() # 查看变换后的结构。

# model_df=features_df.select('features1','price')  # 构建用于线性回归的数据模型

# 5-将数据划分为 训练数据和预测数据

train_df,test_df=df66.randomSplit([0.7,0.3])  # 训练数据和预测数据的比例为 7 比 3

print((train_df.count(), len(train_df.columns)))

print((test_df.count(), len(test_df.columns)))

# 6-构建线性回归模型

from pyspark.ml.regression import LinearRegression  # 导入线性回顾库

print('----------  构建线性回归模型  ----------')

lin_Reg=LinearRegression(labelCol='price')　　# labelCol,相对于 featrues 列，表示要进行预测的列

lr_model=lin_Reg.fit(train_df)　　# 训练数据 ，fit 返回一个 fitted model，即 LineRegressionModel 对象

print('{}{}'.format('方程截距:',lr_model.intercept))　　# intercept 线性方程的截距。

print('{}{}'.format('方程参数系数:',lr_model.coefficients))　# 回归方程中的,变量参数 ,这里分别对应 var_1,var_2,var_3,var_4,var_5

training_predictions=lr_model.evaluate(train_df)　　# 查看预测数据

print('{}{}'.format('误差差值平方:',training_predictions.meanSquaredError))　　# 误差值差值平方

print('{}{}'.format('判定系数：',training_predictions.r2 ))　# r2 判定系数,用来判定，构建的模型是否能够准确的预测,越大说明预测的准确率越高

# 7-使用预测数据,用已经到构建好的预测模型 lr_model
test_results=lr_model.evaluate(test_df)

print(test_results.r2)　　# 查看预测的拟合程度
print(test_results.meanSquaredError)

```
方程截距:-4.290525876672767e-06
方程参数系数:[0.999999993218744,1.0965236688565357e-06,-2.8300055947189365e-08,1.2935310154855614e-06,3.063866936565387e-08,1.2536952365404556e-06,-8.386040468587904e-
误差差值平方:4.79512301478413e-12
判定系数：0.999999999999998
0.999999999999998
4.714390173271384e-12

Process finished with exit code 0
```

2.对二手房进行推荐

首先讲表中的所需数据提取出来

(1) 表中的标题

```
1,宝城一村！配套成熟！全明户型！业主置换！看房方便！急售！
2,南北通透 满五  住房 电梯房 3室2厅 楼层好 视野无遮挡
3,繁荣安居 2室1厅 91.12平
4,康桥 2室2厅1卫 81.11平米 415.00万元
5,进地铁，南北通，步梯中间楼层，低于市价出售，周浦繁华地段
6,急售低于市场60 一梯两户 全明户型 满五年 无说 位置安静
7,地铁房 龙湖品质 送产权车位 满五 税少 位置好 看房随时
8,地铁口直降50万，另外附赠20万产权车位，接受8个月置换
9,虹桥机场新村 满五  住房 南北通透 2室2厅 居家自住 保
10,一手动迁 满五税费少 南北通 中间楼层采光好 有钥匙
11,实勘房源 地铁口 精装两房 配套成熟 接通便利 芙蓉新村诚售
12,九号线700米 南北通户型中间楼层 户型正气
13,本周必卖 看房方便 靠近东站 交通便利 视野开阔 价格可谈
14,小区临河位置， 居家大两房 ，改好的房 满五税少 ，看房随时
15,必卖好房！送产权车位 户型正气 位置好 业主非常诚心出售
16,安静祥和 花园小区 温暖舒心
17,接受长期置换，家具家电全送拎包入住，环境舒适品质小区可谈
18,朝南 满五 沉香苑一街坊 2室1厅 看房方便 有钥匙
19,精装修 南北通透  房东急置换 诚意买
20,满5年 ，周浦好地段，18号地铁周浦万达站，近小海步行街
21,汇丽苑 傅雷  满五年税费少 业主置换 急售！！！
22,正南 2室2厅  星晓家园
23,新华小区，双南两室，全明户型，精装自住，近地铁，业主诚售！
24,业主新挂牌出售      已签 放心推广实勘精选品质好房急售
25,双南电梯两房 满五税少 接受置换 近地铁8.6双轨 景观楼层
26,税费各付 带产权车位 .接受半年置换.看房有钥匙看中价还可谈
27,新上品质小区，近地铁口，毛坯两房，烫金楼层，出行方便，已核验
28,精装修低楼层，拎包入住，税费少，电梯房，交通方便看房随时
29,成熟地段，配套齐全，交通便利，看房随时，性价比高，值得出手
30,朝南 电梯房 90平 2室1厅 简单装修 看房方便 有钥匙
31,主卧朝南，小区中心位置，采光充足，中式风格
32,绿化面积大，采光无遮挡，南面就是公园，休闲生活好位置
33,笋盘推荐！带入室花园（精装2房，全明设计）买进价高，低总价！
```

(2) 表中的中介

```
1,桑春艳
2,巴桂宝
3,胡安洋
4,邹建雄
5,牟必均
6,李晓
7,谢宁
8,洪琼金
9,刘伟
10,閻兴宇
11,刘丽
12,潘序天
13,吴珍
14,魏进
15,丁振祥
16,施海燕
17,洪俊婷
18,余勇
19,蔡礼根
20,侣娜
21,卢刚
22,陈自豪
23,熊烽
24,盘伟
25,卞光伟
26,关佳
27,罗波
28,李海涛
29,陈孟寒
30,王莹莹
31,邱振中
32,沈晓阳
33,费良花
```

(3) 表中的评分

```
1,2,4.2
2,3,4.7
3,4,4.3
4,5,4.5
5,6,4.7
6,7,4.4
7,8,4.3
8,9,3.2
9,10,3.9
10,11,4.6
11,12,4.9
12,13,4.3
13,14,4.6
14,15,4.8
15,16,4.3
16,17,3.8
17,18,4.7
18,19,4.2
19,20,2.6
20,21,4.5
21,22,4.1
22,23,4.4
23,24,5
24,25,3.7
25,26,4.7
26,27,4.9
27,28,4.6
28,29,4.3
29,30,4.3
30,31,4.4
31,32,4.9
32,33,4.5
33,34,4.5
```

## 该挖掘原理

推荐引擎是最常见的机器学习应用，我们可以在各大购物网站上看见这方面的应用。

Spark MLlib 支持 ALS（Alternating Least Squares）推荐算法，是机器学习的协同过滤推荐算法。机器学习的协同过滤推荐算法通过观察所有用户给产品的评价来推断每个用户的喜好，并向每个用户分别推荐多个合适的产品，也可以把某个产品推荐给多个用户。

import pymysql

from pyspark import SparkContext

pymysql.install_as_MySQLdb()

sc=SparkContext("local[*]","ALS")

rawUserData = sc.textFile("/rating.csv")

rawUserData.count()

header = rawUserData.first()

rawUserData = rawUserData.filter(lambda x:x !=header)

print(rawUserData.count())

from pyspark.mllib.recommendation import Rating

rawRatings = rawUserData.map(lambda line:line.split(","))

print(rawRatings.take(5))

ratingsRDD = rawRatings.map(lambda x:(x[0],x[1],x[2]))

print(ratingsRDD.take(5))

numRatings = ratingsRDD.count()

print(numRatings)

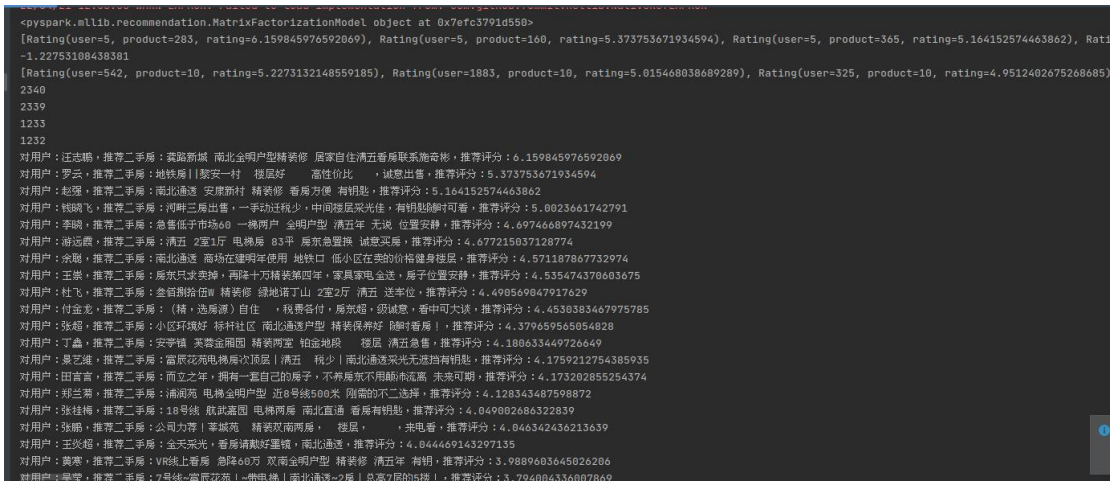
numUsers = ratingsRDD.map(lambda x:x[1]).distinct().count()

print(numUsers)

numproducts = ratingsRDD.map(lambda x:x[0]).distinct().count()

```
print(numproducts)
from pyspark.mllib.recommendation import ALS
model = ALS.train(ratingsRDD,10,10,0.01)
print(model)
print(model.recommendProducts(5,100))
print(model.predict(7,100))
print(model.recommendUsers(product=10,num=5))
itemRDD = sc.textFile("/title.csv")
print(itemRDD.count())
header1 = itemRDD.first()
houseTitle = itemRDD.filter(lambda x:x !=header1)
Title=houseTitle.map(lambda line:line.split(",")).map(lambda a:(float(a[0]),a[1])).collectAsMap()
print(len(Title))
itemRDD1 = sc.textFile("/zhongjie.csv")
print(itemRDD1.count())
header2 = itemRDD1.first()
zhongjie = itemRDD1.filter(lambda x:x !=header2)
yonghu=zhongjie.map(lambda line:line.split(",")).map(lambda a:(float(a[0]),a[1])).collectAsMap()
print(len(yonghu))
recommendP = model.recommendProducts(5,100)
for p in recommendP:
    print("对用户："+yonghu[p[1]]+"，推荐二手房："+str(Title[p[1]])+"，推荐评分："+str(p[2]))
```

结果如下：



最后一个阶段是预测数据阶段，打印出每一行数据的特征值、标签值以及预测值，并通过预测值与真实值进行相减来判断是否预测正确，并把误差打印出来。

# 3.项目总结

## 3.1.项目展望

需要说明的是，由于时间和研究能力的限制，本文还存在很多需要进一步研究和完善的问题，主要包括:

(1)本文所爬取的数据量有限，且没有考虑供房房主发布的时间，后期可考虑增加数据量，并将时间因素加入到模型中，使预测的结果更精确，且具有时效性。

(2)由于前期数据分析时，没有对指标分析彻底，只是做了简单的数量统计。

(3)本文对所爬取的数据分析还是不够彻底，只是浅显的分析了价格、房屋面积等因素，没有对更深层次，如房屋朝向、小区位置所在、周围环境等因素做出分析，故本文存在局限性，还需更进一步完善。

## 3.2.小组总结

我们组的项目名称是基于 58 同城的上海二手房决策分析报告析，这次的任务我们是分工合作的，每个人负责其中的一部分。在作项目的过程中，大家会相互帮助，比如谁遇到问题没办法解决，会有另外一个人来帮助解决，除了分工合作外，我们还积极沟通项目的进展，一个团队最重要的就是合作与沟通，只有沟通好了，项目才能更快地进行。

在整个作项目的过程中，因为有上一学期的基础，和这一学期的不断提高和延伸，从整体来看，完成项目的过程之中，并没有遇见不可逾越的困难，但是在实际操作中，还是有对一些细节把控的不太好，比如在数据清洗方面，在一开始时并没有完全考虑到一些数据可能产生的影响，导致了反复进行好几次的数据清洗工作。通过这一次的实践，我组成员都有这样一个感觉，我们学习不仅要学习理论知识，还要提高动手实践能力，更重要的是能够灵活运用知识，从实训最初的笨手笨脚，到现在可以熟悉开发的流程，这与每个人的努力是分不开的，实训，教会了我们很多的东西，同时也锻炼了大家踏实、稳重和灵活运用以及动手能力的能力。

# 4.源码附录

1.1 数据的采集源代码：

```
import time
from urllib import request
```

```
from urllib.parse import *

import requests
from bs4 import BeautifulSoup
import gzip
import json
from io import BytesIO
import string
import random
import os

headers = [
    "Mozilla/5.0 (Windows NT 6.1; Win64; rv:27.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.36"
    "Mozilla/5.0 (Windows NT 10.0; WOW64; rv:27.0) Gecko/20100101 Firfox/27.0"
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.36"
    "Mozilla/5.0 (Windows NT 10.0; WOW64; rv:10.0) Gecko/20100101 Firfox/10.0"
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/21.0.1180.110 Safari/537.36"
    "Mozilla/5.0 (X11; Ubuntu; Linux i686 rv:10.0) Gecko/20100101 Firfox/27.0"
    "Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/537.1 (KHTML, like Gecko) Chrome/34.0.1838.2 Safari/537.36"
    "Mozilla/5.0 (X11; Ubuntu; Linux i686 rv:27.0) Gecko/20100101 Firfox/27.0"
    "Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.36"
    'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/55.0.2883.87 Safari/537.36'
]


def get_content(url, headers, host):
    # @url：需要登录的网址
    # @headers：模拟的登陆的终端
    # **********************模拟登陆获取网址********************

    random_header = random.choice(headers)
    req = request.Request(url)
    req.add_header("User-Agent", random_header)
    req.add_header("Host", host)
    req.add_header("Accept",
"text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9")
```

```
    # req.add_header("Accept","*/*")
    req.add_header("Accept-Encoding", "gzip,deflate,br")
    req.add_header("Accept-Language", "zh-CN,zh;q=0.9")
    # req.add_header("Connection", "keep-alive")
    req.add_header("Cookie", 'f=n; commontopbar_new_city_info=2%7C%E4%B8%8A%E6%B5%B7%7Csh; userid360_xml=70ADB45668CA26707AE500336729E4D4; time_create=1651904061865; f=n; commontopbar_new_city_info=2%7C%E4%B8%8A%E6%B5%B7%7Csh; commontopbar_ipcity=xx%7C%E6%96%B0%E4%B9%A1%7C0; SECKEY_ABVK=trDSUUgc7IpE9v1ah5wWz8Q2mS6uCh4ZDYob94N7pxQ%3D; BMAP_SECKEY=XjThuhxn0KYn2uMszfPZSE2j37N7Ps2m28uYwjTVZfcMBDGZWTTw9xG6-OfgZWRuw4Ihf3trQf-fDe9Npgc_3obDryDHmpSTdjhJomQejd2mKA99L2dD5HGfC6qgrkOZp6BCE0w1Y7kf5Dmsz6NVXsRGy1p6u1Qn6bIowiiJFB6ypBFP10il6wrrPIQyPaLm; city=bj; id58=CocN02JOXMNAycxTKWC0Ag==; 58tj_uuid=c39662d8-c716-46b9-a8d7-0bc59e39b060; 58home=xx; myfeet_tooltip=end; wmda_new_uuid=1; wmda_uuid=413d1b5fe5b319133ad8079013a8047d; wmda_visited_projects=%3B11187958619315; als=0; xxzl_deviceid=l0v7S2iZ99PD5xEgy4ipXhepeLvoGzAGcXs4%2FM47vw4iOs5w%2BD9A30HlnxZziVCh; Hm_lvt_3f405f7f26b8855bc0fd96b1ae92db7e=1649302732,1649312053; aQQ_ajkguid=560804E1-F053-4081-ADE1-996B2C3A65BF; sessid=69044730-EFA2-4018-A00C-6B742090F518; ajk-appVersion=; fzq_h=41fe926928513e582e552d653b13cca6_1649312050308_5eae8956b6e047d18be5869abded9116_3659345452; init_refer=https%253A%252F%252Fsh.58.com%252Fershoufang%252Fp1%252F; new_uv=5; utm_source=; spm=; 58_ctid=2; is_58_pc=1; commontopbar_new_city_info=11%7C%E4%B8%8A%E6%B5%B7%7Csh; ctid=2; www58com="UserID=64268387810830&UserName=4g7a7be2n"; 58cooper="userid=64268387810830&username=4g7a7be2n"; 58uname=4g7a7be2n; passportAccount="atype=0&bstate=0"; new_session=0; PPU=UID=64268387810830&UN=4g7a7be2n&TT=616d5236b012978391695b015c517803&PBODY=T0vT9TFzKT53C3yEfFMT5zlE6JwqujyH1RmudZL0t-nfZDX4_Dp76IrIvViY18uFPNhJMi29eJLQ0uIlyAGd5R2tNG82uFdtQiUq6z9j88pLc79wilBPtfPK6TrC5VdEToca4bgbo9vQflB8dSxhgEdRzGb-4cc-zB4uKofxYnQ&VER=1&CUID=E_osZYpwVo00_IqXsL8Rbw; xxzl_cid=1ebb0915e5e64a6eb87a939551c70600; xzuid=498044a4-7401-4799-a2f7-26fc6b8d3c01')
    try:
        response = request.urlopen(req)
        contents = response.read()
        if isinstance(contents, bytes):
            buff = BytesIO(contents)
            f = gzip.GzipFile(fileobj=buff)
            contents = f.read()
            contents = contents.decode('utf-8')
        else:
```

```
            print('service mysql restart')
        print('OK! ! ! ')
        return (contents)
    except Exception as e:
        print(e)


def get_links_from(page):
    # @page：表示第几页信息
    # @urls：所有列表的超链接，即子页网址
    # ****************此网站需要模拟登陆**********************
    # 返回全部子网页地址
    # https://search.51job.com/list/000000,000000,0000,00,9,99,+,2,11.html
    urls = []
    path = 'E:/zhaopin/2222/'
    if os.path.exists(path) == False:
        os.makedirs(path)
    for i in range(1, page):
        url = 'https://sh.58.com/ershoufang/p{}'.format(i)

        # url = quote(url, safe=string.printable)  data-v-28819723 data-v-d4f07c36
        info = get_content(url, headers, "sh.58.com")
        info = info.replace(" data-v-28819723","")
        info = info.replace(" data-v-d4f07c36", "")

        if info != None:
            with open(path + str(i) + ".html", 'w') as f:
                f.write(info)
        time.sleep(5)

if __name__ == '__main__':
    # get_recuite_info(2)
    get_links_from(100)
```

2.1 数据清洗源代码：

第一次清洗：

```
def get_html_info(html):
    '''''
    ****************获取网页的有用信息并保存成字典形式***************
    '''
    try:
```

```
        datalist = []
        soup = BeautifulSoup(html, "lxml")  # 设置解析器为“lxml”
        content_list = soup.select("section.list")[0]
        contents = content_list.select("div.property-content")
        for content in contents:
            title = content.select("div.property-content-title h3")[0]
            print(title.text.strip())
            huxing = content.select("div.property-content-info p")[0]
            print(huxing.text.strip())
            mianji = content.select("div.property-content-info p")[1]
            print(mianji.text.strip())
            cengshu = content.select("div.property-content-info p")[3]
            print(cengshu.text.strip())
            fangling = content.select("div.property-content-info p")[4]
            print(fangling.text.strip())
            weizhi = content.select("div.property-content-info-comm p")[1]
            print(weizhi.text.strip())
            zhongjie = content.select("div.property-extra-wrap span")[0]
            print(zhongjie.text.strip())
            pingfen = content.select("div.property-extra-wrap span")[1]
            print(pingfen.text.strip())
            # zhongjiegongsi = content.select("div.property-extra-wrap span")[2]
            # if zhongjiegongsi.text:
            #     zj = zhongjiegongsi.text.strip()
            # else:
            #     zj = "null"
            # print(zj)
            zongjia = content.select("div.property-price p")[0]
            print(zongjia.text.strip())
            jiage = content.select("div.property-price p")[1]
            print(jiage.text.strip())


            data = {
                "标题": title.text.strip(),
                "户型": huxing.text.strip(),
                "面积": mianji.text.strip(),
                "层数": cengshu.text.strip(),
                "房龄": fangling.text.strip(),
                "位置": weizhi.text.strip(),
                "中介": zhongjie.text.strip(),
                "评分": pingfen.text.strip(),
                # "中介公司": zj,
                "总价": zongjia.text.strip(),
```

```
                "价格": jiage.text.strip(),

            }
            datalist.append(data)
        return (datalist)
    except Exception:
        pass
```

第二次清洗：

```
from bs4 import BeautifulSoup
import pandas as pd
import os
import re


def get_html_info(html):
    '''''
    *****************获取网页的有用信息并保存成字典形式****************
    '''
    try:
        datalist = []
        soup = BeautifulSoup(html, "lxml")  # 设置解析器为“lxml”
        content_list = soup.select("section.list")[0]
        contents = content_list.select("div.property-content")
        for content in contents:

            huxing = content.select("div.property-content-info")[0].select("p")[0]
            hx1=huxing.text.strip().replace(' ','').split('室')[0]
            hx2 = huxing.text.strip().replace(' ','').split('室')[1].split('厅')[0]
            hx3 = huxing.text.strip().replace(' ','').split('室')[1].split('厅')[1][0:1]
            mianji = content.select("div.property-content-info")[0].select("p")[1]
            print(mianji.text.strip())
            cengshu = content.select("div.property-content-info")[0].select("p")[3]
            print(cengshu.text.strip().split('(')[0][0:1])
            if cengshu.text.strip().split('(')[0][0:1]=='共':
                cengshu = '其他'
            else:
                cengshu=cengshu.text.strip().split('(')[0]
```

```
            print(cengshu)
            fangling = content.select("div.property-content-info")[0].select("p")[4]
            print(fangling.text.strip())
            weizhi = content.select("div.property-content-info-comm p")[1]
            print(weizhi.text.strip())
            zongjia = content.select("div.property-price p")[0]
            print(zongjia.text.strip())


            data = {
                "price": zongjia.text.strip().replace(" 万", ""),
                "room": hx1,
                "living": hx2,
                "rest": hx3,
                "area": mianji.text.strip().replace("㎡",""),
                "floor": cengshu,
                "age": fangling.text.strip().replace("年建造",""),
                "location": weizhi.text.strip()[0:2]
            }
            datalist.append(data)
        return (datalist)
    except Exception:
        pass

    # 将全部信息保存成 DataFrame 形式，去除无用信息


def get_htmls_all_info(path):
    df = pd.DataFrame({
        "price": [],
        "room": [],
        "living": [],
        "rest": [],
        "area": [],
        "floor": [],
        "age": [],
        "location": []
    })
    dirs = os.listdir(path)
    for dir in dirs:
        # print(dir)
        if dir.find('swp'):
            dir = dir.strip('.' and '.swp')
```

```
        p = os.path.join(path, dir)
        html = open(p, encoding='gbk').read()
        data = get_html_info(html)
        print(data)
        df = df.append(data, ignore_index=True)
    return df


if __name__ == '__main__':
    path = 'D:/zhaopin/2222'
    df = get_htmls_all_info(path)
    df.to_csv('D:/zhaopin/bigdata4', index=False,sep='\t')
    print(df)
```

3.数据分析、导出与迁移：

```
from bs4 import BeautifulSoup
import pandas as pd
import os
import re


def get_html_info(html):
    '''''
    *****************获取网页的有用信息并保存成字典形式****************
    '''
    try:
        datalist = []
        soup = BeautifulSoup(html, "lxml")  # 设置解析器为“lxml”
        content_list = soup.select("section.list")[0]
        contents = content_list.select("div.property-content")
        for content in contents:

            huxing = content.select("div.property-content-info")[0].select("p")[0]
            hx1=huxing.text.strip().replace(' ','').split('室')[0]
            hx2 = huxing.text.strip().replace(' ','').split('室')[1].split('厅')[0]
            hx3 = huxing.text.strip().replace(' ','').split('室')[1].split('厅')[1][0:1]
            mianji = content.select("div.property-content-info")[0].select("p")[1]
            print(mianji.text.strip())
```

```
            cengshu = content.select("div.property-content-info")[0].select("p")[3]
            print(cengshu.text.strip().split('(')[0][0:1])
            if cengshu.text.strip().split('(')[0][0:1]=='共':
                cengshu = '其他'
            else:
                cengshu=cengshu.text.strip().split('(')[0]
            print(cengshu)
            fangling = content.select("div.property-content-info")[0].select("p")[4]
            print(fangling.text.strip())
            weizhi = content.select("div.property-content-info-comm p")[1]
            print(weizhi.text.strip())
            zongjia = content.select("div.property-price p")[0]
            print(zongjia.text.strip())


            data = {
                "price": zongjia.text.strip().replace(" 万", ""),
                "room": hx1,
                "living": hx2,
                "rest": hx3,
                "area": mianji.text.strip().replace("㎡",""),
                "floor": cengshu,
                "age": fangling.text.strip().replace("年建造",""),
                "location": weizhi.text.strip()[0:2]
            }
            datalist.append(data)
        return (datalist)
    except Exception:
        pass

    # 将全部信息保存成 DataFrame 形式，去除无用信息


def get_htmls_all_info(path):
    df = pd.DataFrame({
        "price": [],
        "room": [],
        "living": [],
        "rest": [],
        "area": [],
        "floor": [],
        "age": [],
```

```
        "location": []
    })
    dirs = os.listdir(path)
    for dir in dirs:
        # print(dir)
        if dir.find('swp'):
            dir = dir.strip('.' and '.swp')
        p = os.path.join(path, dir)
        html = open(p, encoding='gbk').read()
        data = get_html_info(html)
        print(data)
        df = df.append(data, ignore_index=True)
    return df


if __name__ == '__main__':
    path = 'D:/zhaopin/2222'
    df = get_htmls_all_info(path)
    df.to_csv('D:/zhaopin/bigdata4', index=False,sep='\t')
    print(df)


from sqlalchemy import create_engine

from pyspark import SparkContext
from pyspark.sql import SQLContext
import pymysql
pymysql.install_as_MySQLdb()

import pandas as pd
# from pyspark.shell import sc,sqlContext
sc = SparkContext("local[*]")
sqlContext = SQLContext(sc)
RDD1 = sc.textFile("file:/data/bigdata1")
header = RDD1.first()
RDD2 = RDD1.filter(lambda x:x != header)
RDD3 = RDD2.map(lambda line: line.split("\t"))

from pyspark.sql import Row

house_Rows = RDD3.map(lambda p:
                      Row(
                             title=p[0],
                             huxing=p[1],
```

```
                            jiage=p[2],
                            mianji=p[3],
                           cengshu=p[4],
                         fangling=p[5],
                         weizhi=p[6],
                         zhongjie=p[7],
                         pingfen=p[8],
                         zongjia=p[9],
                      )
                      )
from pyspark.sql import SQLContext

sqlContext = SQLContext(sc)
house_df = sqlContext.createDataFrame(house_Rows)
house_df.printSchema()
print(house_df.show(5))
df = house_df.alias("df")
print(df.show(5))
print(df.select("mianji").groupby("mianji").count().show())
print("============开始分析数据==============")
sqlContext.registerDataFrameAsTable(df, "house_table")
print(df.select("fangling").groupby("fangling").count().show())
sqlContext.registerDataFrameAsTable(df, "house_table")
print(sqlContext.sql("select * from house_table").show())
df_house = sqlContext.sql("select * from house_table").toPandas()
df_zongjia = sqlContext.sql("""
SELECT
    a.zongjia_qujian AS zongjia,
    count(*) AS num
FROM
    (
        SELECT
            CASE
        WHEN zongjia < 100 THEN
            '100 以下'
        WHEN zongjia < 200 THEN
            '100 - 200'
        WHEN zongjia < 250 THEN
            '200 - 250'
        WHEN zongjia < 300 THEN
            '250 - 300'
        WHEN zongjia < 350 THEN
            '300 - 350'
        WHEN zongjia < 400 THEN
```

```
            '350 - 400'
        WHEN zongjia < 500 THEN
            '400 - 500'
        WHEN zongjia < 800 THEN
            '500 - 800'
        WHEN zongjia < 1000 THEN
            '800 - 1000'
        ELSE
            '1000 以上'
        END AS zongjia_qujian
        FROM
            house_table
    ) AS a
GROUP BY
    a.zongjia_qujian
ORDER BY
    count(*) DESC
""").toPandas()
df_mianji = sqlContext.sql("""
SELECT
    b.mianji_qujian AS mianji,
    count(*) AS num
FROM
    (
        SELECT
            CASE
        WHEN mianji < 50 THEN
            '50 以下'
        WHEN mianji < 60 THEN
            '50 - 60'
        WHEN mianji < 70 THEN
            '60 - 70'
        WHEN mianji < 80 THEN
            '70 - 80'
        WHEN mianji < 90 THEN
            '80 - 90'
        WHEN mianji < 100 THEN
            '90 - 100'
        WHEN mianji < 120 THEN
            '100 - 120'
        WHEN mianji < 1400 THEN
            '120 - 140'
        WHEN mianji < 210 THEN
            '140 - 210'
```

```
        ELSE
            '210 以上'
        END AS mianji_qujian
        FROM
            house_table
    ) AS b
GROUP BY
    b.mianji_qujian
ORDER BY
    count(*) DESC
""").toPandas()
df_jiage = sqlContext.sql("""
SELECT
    d.jiage_qujian AS jiage,
    count(*) AS num
FROM
    (
        SELECT
            CASE
        WHEN jiage < 15000 THEN
            '15000 以下'
        WHEN jiage < 30000 THEN
            '15000 - 30000'
        WHEN jiage < 50000 THEN
            '30000 - 50000'
        WHEN jiage < 80000 THEN
            '50000 - 80000'
        WHEN jiage < 100000 THEN
            '80000 - 100000'
        WHEN jiage < 130000 THEN
            '100000 - 130000'
        WHEN jiage < 150000 THEN
            '130000 - 150000'
        WHEN jiage < 180000 THEN
            '150000 - 180000'
        WHEN jiage < 200000 THEN
            '180000 - 200000'
        ELSE
            '200000 以上'
        END AS jiage_qujian
        FROM
            house_table
    ) AS d
GROUP BY
```

```
    d.jiage_qujian
ORDER BY
    count(*) DESC
""").toPandas()
df_zhongjie = sqlContext.sql("""
SELECT
DISTINCT zhongjie,pingfen
FROM
    house_table
ORDER BY pingfen DESC
""").toPandas()
df_title = sqlContext.sql("""
SELECT title FROM house_table
""").toPandas()
df_fangling = sqlContext.sql("""
SELECT
    a.fangling_qujian AS fangling,
    count(*) AS num
FROM
    (
        SELECT
            CASE
        WHEN fangling < 1970 THEN
            '1970年以前'
        WHEN fangling < 1990 THEN
            '1970 - 1990'
        WHEN fangling < 2000 THEN
            '1990 - 2000'
        WHEN fangling < 2010 THEN
            '2000 - 2010年'
        ELSE
            '2010年以后'
        END AS fangling_qujian
        FROM
            house_table
    ) AS a
GROUP BY
    a.fangling_qujian
ORDER BY
    count(*) DESC
""").toPandas()
df_cengshu = sqlContext.sql("""
SELECT
    a.cengshu,
```

```
    count(*)
FROM
    (
        SELECT
            CASE
        WHEN cengshu IN ('中层', '低层', '高层') THEN
            cengshu
        ELSE
            '其他'
        END AS cengshu
        FROM
            house_table
    ) AS a
GROUP BY
    a.cengshu
""").toPandas()
#创建数据库链接
conn=create_engine('mysql+mysqldb://root:dmq3677...@localhost:3306/house?charset=utf8')
#保存公司规模分布到数据库
pd.io.sql.to_sql(
    df_house,
    name='house',
    con=conn,
    schema='house',
    if_exists='replace',
    index=True,
    index_label='id'
)
pd.io.sql.to_sql(
    df_zongjia,
    name='zongjia',
    con=conn,
    schema='house',
    if_exists='replace',
    index=True,
    index_label='id'
)
pd.io.sql.to_sql(
    df_mianji,
    name='mianji',
    con=conn,
    schema='house',
    if_exists='replace',
```

```
    index=True,
    index_label='id'
)

pd.io.sql.to_sql(
    df_jiage,
    name='jiage',
    con=conn,
    schema='house',
    if_exists='replace',
    index=True,
    index_label='id'
)
pd.io.sql.to_sql(
    df_fenci,
    name='fenci',
    con=conn,
    schema='house',
    if_exists='replace',
    index=True,
    index_label='id'
)
pd.io.sql.to_sql(
    df_fangling,
    name='fangling',
    con=conn,
    schema='house',
    if_exists='replace',
    index=True,
    index_label='id'
)
pd.io.sql.to_sql(
    df_cengshu,
    name='cengshu',
    con=conn,
    schema='house',
    if_exists='replace',
    index=True,
    index_label='id'
)

```

数据可视化

Jieba 分词：

```python
import pandas as pd
import re
import numpy as np
import jieba
import jieba.analyse

import pymysql
from sqlalchemy import create_engine

pymysql.install_as_MySQLdb()
jieba.load_userdict("/home/zhangyu/housezy.dict")
# 加载数据
data = pd.read_csv('/home/zhangyu/house1.csv', header=1, names=["公司全名","公司简称","公司规模","融资阶段","区域","职位名称","工作经验","学历要求","薪资","职位福利","经营范围","职位类型","公司福利","第二职位类型","城市"])
data = data[1:].astype(str)

# 使用 pandas 中的方法,取出岗位描述列中符合的内容
job_des = data.职位福利+" "+data.公司福利
# jieba 分词，并统计词频
zidian = {}
r1 = re.compile(r'\w')  # 使用正则表达式,筛选[A-Za-z0-9_]
r2 = re.compile(r'[^\d]')  # 使用正则表达式,筛选[0-9_]
r4 = re.compile(r'[^_]')
r3 = re.compile(r'[\u4e00-\u9fa5]')  # 使用正则表达式,筛选纯中文
stopkeyword = [line.strip() for line in open('/home/zhangyu/stopword').readlines()]  # 加载停用词
bogs = []  # 用于存储长的英文语句
for i in job_des:
    seg_list = jieba.cut(i)
    for word in seg_list:
        if word not in stopkeyword and r1.match(word) and r2.match(word) and r3.match(word) and r4.match(
                word):  # 筛选语句中的英文
            word = word.lower()  # 小写化所有词
            if len(word) >= 10:
                bogs.append(word)
            elif len(word) != 1 or word in ['c', 'r']:
                # 统计词频
                if word in zidian:
                    zidian[word] += 1
                else:
                    zidian[word] = 1

```

```
                    # 将字典的值逆序
zidian = sorted(zidian.items(), key=lambda item: item[1], reverse=True)
zidian = pd.DataFrame(zidian, columns=['skill_name', 'num'])
zidian.loc[(zidian.skill_name == 'r'), 'skill_name'] = "R"
zidian.loc[(zidian.skill_name == 'c'), 'skill_name'] = "C"
df = zidian[zidian.num > 70]


# 用于分割长语句中的关键词
def cut(sentences, words):
    itr = []
    for st in sentences:
        for word in words:
            if st.find(word) != -1:
                itr.append(word)
                cut(st.split(word), words)
    return itr
# 将分割词的词频累计到‘zidian’中
word_bogs = cut(bogs, df.skill_name)
for word_bog in word_bogs:
    if word_bog in list(zidian.skill_name):
        zidian.loc[zidian.skill_name == word_bog, 'num'] += 1
        # zidian.to_csv('/data/zhilian/job_des')
print(zidian)
#将 zidian 中数据导入到 mysql 数据库中
conn = create_engine('mysql+mysqldb://root:strongs@localhost:3306/house?charset=utf8')
```

```
pd.io.sql.to_sql(zidian, name='house_jieba', con=conn, schema='house', if_exists='append', index=False)
```

5.数据可视化

**Models:**

```
from django.db import models

# Create your models here.
class Mianji(models.Model):
    id = models.BigIntegerField(primary_key=True)
    mianji = models.TextField(db_column='mianji', blank=True, null=True)  # Field name made lowercase.
    num = models.BigIntegerField(db_column='num',blank=True, null=True)

    # class Meta:
    #     managed = False
```

```
    #     db_table = 'company_size'
class Zongjia(models.Model):
   id = models.BigIntegerField(primary_key=True)
   zongjia = models.TextField(db_column='zongjia', blank=True, null=True)
   num = models.BigIntegerField(db_column='num', blank=True, null=True)

class Fangling(models.Model):
    id = models.BigIntegerField(primary_key=True)
    fangling = models.TextField(db_column='fangling', blank=True, null=True)
    num = models.BigIntegerField(db_column='num', blank=True, null=True)

class Cengshu(models.Model):
    id = models.BigIntegerField(primary_key=True)
    cengshu = models.TextField(db_column='cengshu', blank=True, null=True)
    num = models.BigIntegerField(db_column='num', blank=True, null=True)

class Fenci(models.Model):
    Fenci = models.TextField(db_column='Fenci', blank=True, null=True)
    num = models.IntegerField(db_column='num', blank=True, null=True)

class Jiage(models.Model):
    id = models.BigIntegerField(primary_key=True)
    Jiage = models.TextField(db_column='jiage', blank=True, null=True)
    num = models.BigIntegerField(db_column='num', blank=True, null=True)
```

**views:**

```
import json

from django.http import HttpResponse
from django.shortcuts import render

from web.models import Mianji, Zongjia,Fangling, Cengshu, Fenci, Jiage


def mianji_data(request):
    data_dict = []
    items = Mianji.objects.all()
    for item in items:
        data_dict.append({
            'name':item.mianji,
            'value':item.num,

        })
```

```
    return HttpResponse(json.dumps(data_dict))

def mianji(request):
    return render(request,'mianji.html')

def zongjia_data(request):
    data_dict = []
    items = Zongjia.objects.all()
    for item in items:
        data_dict.append({
            'name':item.zongjia,
            'value':item.num,
        })
    return HttpResponse(json.dumps(data_dict))

def zongjia(request):
    return render(request,'zongjia.html')

def fangling_data(request):
    data_dict = []
    items = Fangling.objects.all()
    for item in items:
        data_dict.append({
            'name':item.fangling,
            'value':item.num,
        })
    return HttpResponse(json.dumps(data_dict))

def fangling(request):
    return render(request, 'fangling.html')

def cengshu_data(request):
    data_dict = []
    items = Cengshu.objects.all()
    for item in items:
        data_dict.append({
            'name':item.cengshu,
            'value':item.num,
        })
    return HttpResponse(json.dumps(data_dict))
def cengshu(request):
    return render(request,'cengshu.html')


```

```
62.
63. def fenci_data(request):
64.     data_dict = []
65.     items = Fenci.objects.filter(num__gte=150)
66.     for item in items:
67.         data_dict.append({
68.             'name':item.Fenci,
69.             'value':item.num,
70.         })
71.     return HttpResponse(json.dumps(data_dict,ensure_ascii=False))
72. def fenci(request):
73.     return render(request,'word_cloud.html')
74.
75. def jiage_data(request):
76.     data_dict = []
77.     items = Jiage.objects.all()
78.     for item in items:
79.         data_dict.append({
80.             'name':item.Jiage,
81.             'value':item.num,
82.         })
83.     return HttpResponse(json.dumps(data_dict))
84. def jiage(request):
85.     return render(request,'jiage.html')
86.
87. # Create your views here.
```

**urls:**

```
1.  """house URL Configuration
2.
3.  The `urlpatterns` list routes URLs to views. For more information please see:
4.      https://docs.djangoproject.com/en/3.2/topics/http/urls/
5.  Examples:
6.  Function views
7.      1. Add an import:  from my_app import views
8.      2. Add a URL to urlpatterns:  path('', views.home, name='home')
9.  Class-based views
10.     1. Add an import:  from other_app.views import Home
11.     2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
12. Including another URLconf
13.     1. Import the include() function: from django.urls import include, path
14.     2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
15. """
```

```
from django.contrib import admin
from django.urls import path

from web import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('mianji_data/', views.mianji_data),
    path('mianji/', views.mianji),
    path('zongjia_data/', views.zongjia_data),
    path('zongjia/', views.zongjia),
    path('fangling_data/', views.fangling_data),
    path('fangling/', views.fangling),
    path('cengshu_data/', views.cengshu_data),
    path('cengshu/', views.cengshu),
    path('fenci_data/', views.fenci_data),
    path('fenci/', views.fenci),
    path('jiage_data/', views.jiage_data),
    path('jiage/', views.jiage),

]
```

6.数据挖掘分析

```
对上海二手房价格预测
from pyspark import SparkContext
from pyspark.sql import SQLContext
from pyspark.ml.feature import  StringIndexer
import pandas as pd
from pyspark.sql.functions import col
import pyspark.sql.types
from pyspark.sql.functions import udf
from pyspark.ml.feature import  VectorAssembler
from pyspark.ml.regression import LinearRegression
sc = SparkContext("local[*]")
sqlContext = SQLContext(sc)


row_df = sqlContext.read.format("csv") \
.option("header", "true") \
.option("delimiter", "\t") \
.load("hdfs://localhost:9000/bigdata4.csv")
print (row_df.count())
print(row_df.printSchema())
```

```
19. row_df.select('room','living','rest','area','floor','age','location','price')
.show(10)

20. df= row_df.select(
21. ['floor','location','age']+
22. [col(column).cast("double").alias(column)
23. for column in row_df.columns[0:5] ] )


24. print (df.printSchema())
25. print(df.select('room','living','rest','area','floor','age','location','pric
e').show(10))
26. train_df, test_df = df.randomSplit([0.7, 0.3])
27. train_df.cache()
28. test_df.cache()
29. floorIndexer = StringIndexer(
30. inputCol='floor',
31. outputCol="floor_Index")
32. floorTransformer=floorIndexer.fit(df)
33. for i in range(0,len(floorTransformer.labels)):
34. print (str(i)+':'+floorTransformer.labels[i])
35. df1=floorTransformer.transform(df)
36. print (df1.columns)
37. df1.select("floor","floor_Index").show(10)
38. from pyspark.ml.feature import  OneHotEncoder
39. encoder = OneHotEncoder(dropLast=False,
40. inputCol='floor_Index',
41. outputCol="floor_IndexVec")
42. df2=encoder.transform(df1)
43. print (df2.columns)
44. df2.select("floor","floor_Index",
45. "floor_IndexVec").show(10)


46. # assemblerInputs =['price', 'room', 'living', 'rest', 'area','floor_IndexVec']
47. # print (assemblerInputs)
48. # assembler = VectorAssembler(
49. #                           inputCols=assemblerInputs,
50. #                           outputCol="features")
51. # df22=assembler.transform(df2)
52. # print (df22.columns)
53. # df22.select('features').show(1)

54. locationIndexer = StringIndexer(
```

```
55. inputCol='location',
56. outputCol="location_Index")
57. locationTransformer=locationIndexer.fit(df2)
58. for i in range(0,len(locationTransformer.labels)):
59. print (str(i)+':'+locationTransformer.labels[i])
60. df3=locationTransformer.transform(df2)
61. print (df3.columns)
62. df3.select("location","location_Index").show(10)
63. from pyspark.ml.feature import  OneHotEncoder
64. encoder = OneHotEncoder(dropLast=False,
65. inputCol='location_Index',
66. outputCol="location_IndexVec")
67. df4=encoder.transform(df3)
68. print (df4.columns)
69. df4.select("location","location_Index",
70. "location_IndexVec").show(10)
71. #
72. # assemblerInputs =['location_IndexVec','features']
73. # print (assemblerInputs)
74. # assembler = VectorAssembler(
75. #                          inputCols=assemblerInputs,
76. #                          outputCol="features1")
77. # df44=assembler.transform(df4)
78. # print (df44.columns)
79. #
80. #
81. #
82. ageIndexer = StringIndexer(
83. inputCol='age',
84. outputCol="age_Index")
85. ageTransformer=ageIndexer.fit(df4)
86. for i in range(0,len(ageTransformer.labels)):
87. print (str(i)+':'+ageTransformer.labels[i])
88. df5=ageTransformer.transform(df4)
89. print (df5.columns)
90. df5.select("age","age_Index").show(10)
91. from pyspark.ml.feature import  OneHotEncoder
92. encoder = OneHotEncoder(dropLast=False,
93. inputCol='age_Index',
94. outputCol="age_IndexVec")
95. df6=encoder.transform(df5)
96. print (df6.columns)
97. df6.select("age","age_Index",
98. "age_IndexVec").show(10)
```

```
assemblerInputs =[ 'price','room', 'living', 'rest', 'area','floor_IndexVec','location_IndexVec','age_IndexVec']
print (assemblerInputs)

assembler = VectorAssembler(
inputCols=assemblerInputs,
outputCol="features")
df66=assembler.transform(df6)
print (df66.columns)
df66.select('features').show(10)



# 3-探索分析数据
print('-------------- 探索分析数据 -----------------')
print((df66.count(), len(df66.columns)))  # 查看数据规模
df66.printSchema()  # 查看数据结构类型
df66.describe().show(5,False)  # 查看数据集的统计数据，包括平均值，标准差，数量统计等。
from pyspark.sql.functions import corr
# df.select(corr('var_1','output')).show()  # 计算数据方差

# 4-数据转换,适应模型算法中的要求
from pyspark.ml.linalg import Vector
from pyspark.ml.feature import VectorAssembler  # 导入库 VectorAssembler

# print('-------------- 数据转换 ------------------')
# vec_assmebler=VectorAssembler(inputCols=['price'],outputCol='features1') # 转换，这里相对将多元一次方程中的各变量存放到一个向量中
# features_df=vec_assmebler.transform(df66)

# features_df.printSchema() # 查看变换后的结构。

# model_df=features_df.select('features1','price')  # 构建用于线性回归的数据模型

# 5-将数据划分为 训练数据和预测数据
train_df,test_df=df66.randomSplit([0.7,0.3])  # 训练数据和预测数据的比例为 7 比 3

print((train_df.count(), len(train_df.columns)))

print((test_df.count(), len(test_df.columns)))
```

```
# 6-构建线性回归模型

from pyspark.ml.regression import LinearRegression        # 导入线性回顾库

print('-------------- 构建线性回归模型 ------------------')

lin_Reg=LinearRegression(labelCol='price')                 # labelCol,相对于featrues 列，表示要进行预测的列

lr_model=lin_Reg.fit(train_df)                             # 训练数据 ,fit 返回一个 fitted model，即 LineRegressionModel 对象

print('{}{}'.format('方程截距:',lr_model.intercept))        # intercept 线性方程的截距。

print('{}{}'.format('方程参数系数:',lr_model.coefficients))  # 回归方程中的,变量参数 ,这里分别对应 var_1,var_2,var_3,var_4,var_5

training_predictions=lr_model.evaluate(train_df)           # 查看预测数据

print('{}{}'.format('误差差值平方:',training_predictions.meanSquaredError)) # 误差值差值平方

print('{}{}'.format('判定系数：',training_predictions.r2 ))  # r2 判定系数,用来判定，构建的模型是否能够准确的预测,越大说明预测的准确率越高

# 7-使用预测数据,用已经到构建好的预测模型 lr_model
test_results=lr_model.evaluate(test_df)

print(test_results.r2)                             # 查看预测的拟合程度
print(test_results.meanSquaredError)
```

```
1. 对二手房的推荐
2. import pymysql
3. from pyspark import SparkContext
4. pymysql.install_as_MySQLdb()
5. sc=SparkContext("local[*]","ALS")
6. rawUserData = sc.textFile("/rating.csv")
7. rawUserData.count()
8. header = rawUserData.first()
9. rawUserData = rawUserData.filter(lambda x:x !=header)
10. print(rawUserData.count())
11. from pyspark.mllib.recommendation import Rating
12. rawRatings = rawUserData.map(lambda line:line.split(","))
```

```
print(rawRatings.take(5))
ratingsRDD = rawRatings.map(lambda x:(x[0],x[1],x[2]))
print(ratingsRDD.take(5))
numRatings = ratingsRDD.count()
print(numRatings)
numUsers = ratingsRDD.map(lambda x:x[1]).distinct().count()
print(numUsers)
numproducts = ratingsRDD.map(lambda x:x[0]).distinct().count()
print(numproducts)
from pyspark.mllib.recommendation import ALS
model = ALS.train(ratingsRDD,10,10,0.01)
print(model)
print(model.recommendProducts(5,100))
print(model.predict(7,100))
print(model.recommendUsers(product=10,num=5))
itemRDD = sc.textFile("/title.csv")
print(itemRDD.count())
header1 = itemRDD.first()
houseTitle = itemRDD.filter(lambda x:x !=header1)
Title=houseTitle.map(lambda line:line.split(",")).map(lambda a:(float(a[0]),a[1])).collectAsMap()
print(len(Title))
itemRDD1 = sc.textFile("/zhongjie.csv")
print(itemRDD1.count())
header2 = itemRDD1.first()
zhongjie = itemRDD1.filter(lambda x:x !=header2)
yonghu=zhongjie.map(lambda line:line.split(",")).map(lambda a:(float(a[0]),a[1])).collectAsMap()
print(len(yonghu))
recommendP = model.recommendProducts(5,100)
for p in recommendP:
print("对用户："+yonghu[p[1]]+"，推荐二手房："+str(Title[p[1]])+"，推荐评分："+str(p[2]))
```